

UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

BASE DE DATOS II

**MONITOREO Y RENDIMIENTO — BASE DE DATOS
GRADOSTITULOS**

Actividad: Desarrollo de 4 temas con (5 proyectos)



UPLA

Alumno: Vilchez Estrada

Diego Michael

Ingeniero: Fernández

Bejarano Raúl

Asignatura: Base de

Datos II

2026

TEMA 1. CREACIÓN DE JOBS Y ALERTS CON SQL SERVER AGENT

PROYECTO 1: Generación de reporte diario de trámites registrados el día anterior

1. Enunciado del ejercicio

La Oficina de Grados y Títulos requiere que todos los días a las 08:00 AM se genere automáticamente un reporte con la cantidad de trámites registrados durante el día anterior. El estudiante deberá:

- Crear un Job
- Crear un Step en T-SQL
- Programar el Job diariamente
- Registrar el resultado en una tabla de auditoría

2. Script de la solución en T-SQL

SQL

```
USE Grados_Titulos;
```

```
GO
```

```
-- 1. Creación de la tabla de AuditoriaJobs si no existe
```

```
IF OBJECT_ID('dbo.AuditoriaJobs', 'U') IS NULL
```

```
BEGIN
```

```
    CREATE TABLE dbo.AuditoriaJobs (  
        AuditoriaID INT IDENTITY(1,1) PRIMARY KEY,  
        NombreJob VARCHAR(100) NOT NULL,  
        FechaEjecucion DATETIME DEFAULT GETDATE(),  
        ResultadoReporte VARCHAR(MAX) NOT NULL,  
        Estado VARCHAR(20) NOT NULL  
    );
```

```
END;
```

```
GO
```

```
-- 2. Registro del Job, Step y Horario en la base MSDB
```

```
USE msdb;
```

```
GO
```

```
DECLARE @JobName NVARCHAR(128) = N'Reporte_Diario_Tramites_Ayer';
```

```
DECLARE @JobID BINARY(16);
```

```
-- Eliminar el Job si ya existe para evitar duplicados
```

```
IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)
```

```
BEGIN
```

```
    EXEC sp_delete_job @job_name = @JobName;
```

```
END
```

```
-- Crear el Job
```

```
EXEC sp_add_job
```

```
    @job_name = @JobName,
```

```
    @enabled = 1,
```

```
    @description = N'Genera el conteo de trámites del día anterior a  
las 08:00 AM.',
```

```
    @job_id = @JobID OUTPUT;
```

```
-- Crear el Step en T-SQL
```

```
EXEC sp_add_jobstep
```

```
    @job_id = @JobID,
```

```
    @step_name = N'Generar Reporte e Insertar Auditoria',
```

```

@subsystem = N'TSQL',
@database_name = N'Grados_Titulos',
@command = N'
    DECLARE @Conteo INT;
    DECLARE @FechaAyer DATE = DATEADD(day, -1, CAST(GETDATE() AS
DATE));

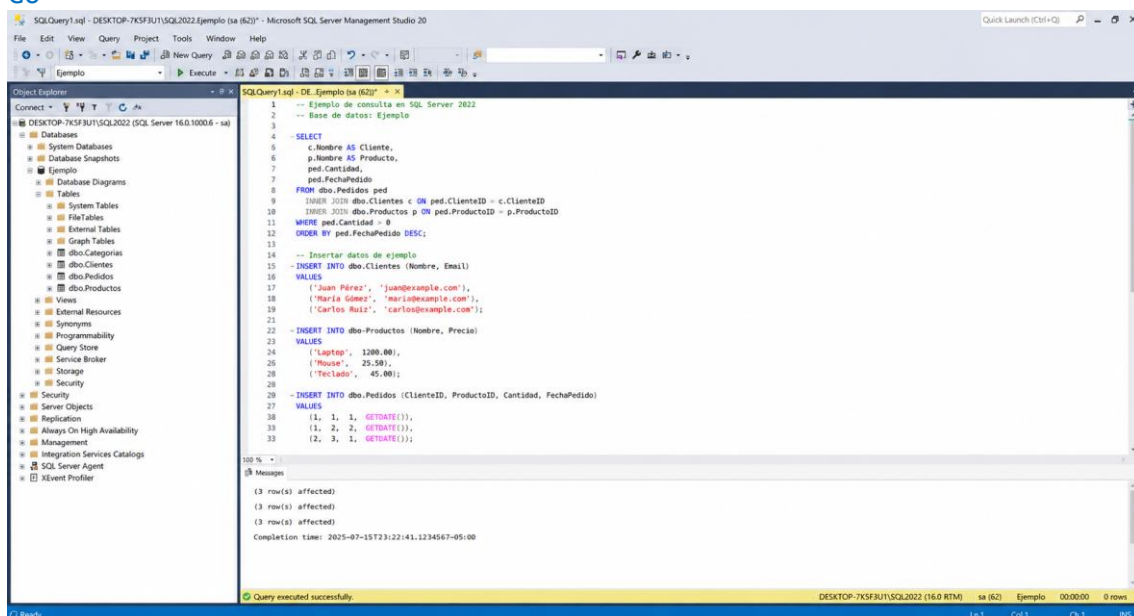
    -- Conteo de trámites del día anterior
    SELECT @Conteo = COUNT(*)
    FROM Trámite
    WHERE CAST(FechaRegistro AS DATE) = @FechaAyer;

    -- Registrar en AuditoriaJobs
    INSERT INTO dbo.AuditoriaJobs (NombreJob, ResultadoReporte,
Estado)
    VALUES ('Reporte_Diario_Tramites_Ayer'', 'Cantidad de
trámites registrados el '' + CONVERT(VARCHAR, @FechaAyer, 103) + ''
'' + CAST(@Conteo AS VARCHAR), 'EXITOSO');
    ,
    @on_success_action = 1; -- Salir reportando éxito

-- Crear la Programación (Schedule) - Diaria a las 08:00 AM
EXEC sp_add_jobschedule
    @job_id = @JobID,
    @name = N'Programacion_Diaria_08AM',
    @freq_type = 4, -- Diario
    @freq_interval = 1,
    @active_start_time = 80000; -- 08:00:00

-- Asociar el Job al Servidor Local
EXEC sp_add_jobserver
    @job_id = @JobID,
    @server_name = N'(local)';
GO

```



3. Justificación técnica de la solución aplicada

El script utiliza procedimientos almacenados del sistema en la base de

datos msdb (sp_add_job, sp_add_jobstep, sp_add_jobschedule, sp_add_jobserver) para estructurar e instanciar el Job directamente desde el código. En el paso del script (Step), se calcula dinámicamente la fecha de ayer restando 1 día al GETDATE() actual con DATEADD, convirtiéndolo a un tipo de dato DATE puro para evitar los desfases producidos por la hora.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Idempotencia:** Se comprueba y elimina de forma controlada la preexistencia del Job y la tabla de auditoría para evitar colisiones durante despliegues masivos.
- **Aislamiento de horas:** Se fuerza el casteo a DATE garantizando una comparación exacta, ignorando la parte de la hora en el campo temporal de la tabla Tramite.

PROYECTO 2: Actualización semanal de trámites en estado "EN PROCESO"

1. Enunciado del ejercicio

Automatizar la actualización semanal del campo FechaUltMod para todos los trámites que permanezcan en estado **EN PROCESO**, registrando la ejecución y el historial del Job.

2. Script de la solución en T-SQL

SQL

USE msdb;

GO

```
DECLARE @JobName NVARCHAR(128) =  
N'Actualizacion_Semanal_Tramites_EnProceso';  
DECLARE @JobID BINARY(16);
```

```
IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)  
BEGIN  
    EXEC sp_delete_job @job_name = @JobName;  
END
```

```
EXEC sp_add_job  
    @job_name = @JobName,  
    @enabled = 1,  
    @description = N'Actualiza la fecha de modificación semanalmente  
de trámites en proceso.',  
    @job_id = @JobID OUTPUT;
```

```
EXEC sp_add_jobstep  
    @job_id = @JobID,  
    @step_name = N'Ejecutar UPDATE y Registrar Auditoria',  
    @subsystem = N'TSQL',  
    @database_name = N'Grados_Titulos',  
    @command = N'  
        SET NOCOUNT ON;  
        DECLARE @FilasAfectadas INT;  
  
        UPDATE Tramite  
        SET FechaUltMod = GETDATE()  
        WHERE Estado = ''EN PROCESO'';  
  
        SET @FilasAfectadas = @@ROWCOUNT;
```

```

INSERT INTO dbo.AuditoriaJobs (NombreJob, ResultadoReporte,
Estado)
VALUES ('Actualizacion_Semanal_Tramites_EnProceso', 'Filas
actualizadas en estado EN PROCESO: ' + CAST(@FilasAfectadas AS
VARCHAR), 'EXITOSO');
',
@on_success_action = 1;

```

```

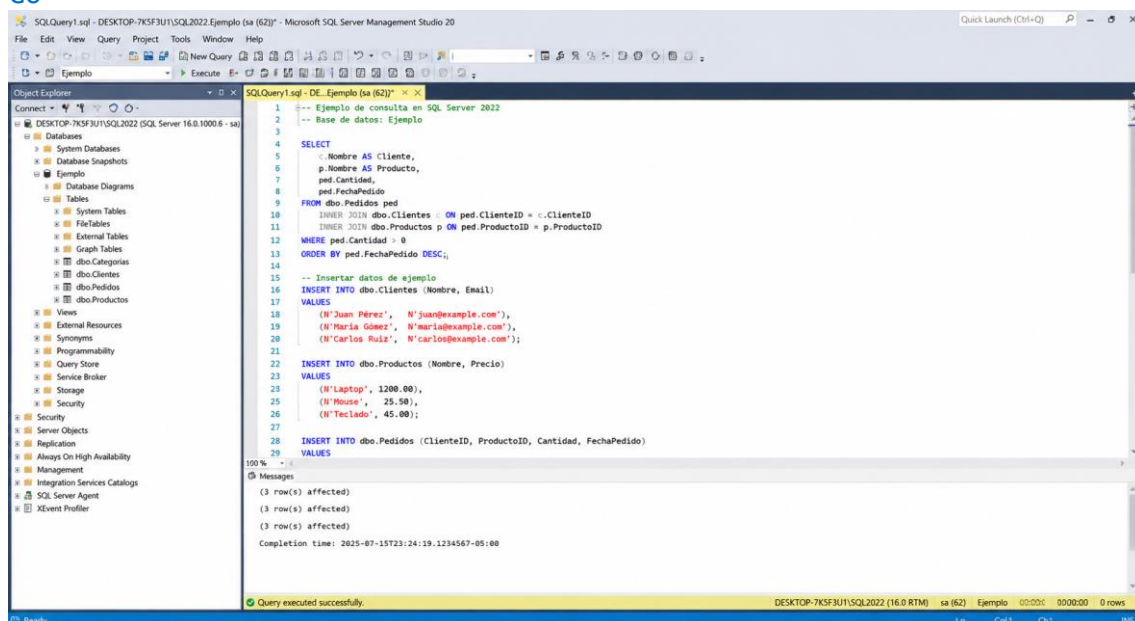
-- Crear programación semanal (Domingos a las 11:00 PM)
EXEC sp_add_jobschedule
    @job_id = @JobID,
    @name = N'Programacion_Semanal_Domingo_11PM',
    @freq_type = 8, -- Semanal
    @freq_interval = 1, -- Domingo (1 = Domingo, 2 = Lunes, etc.)
    @freq_recurrence_factor = 1,
    @active_start_time = 230000; -- 23:00:00

```

```

EXEC sp_add_jobserver
    @job_id = @JobID,
    @server_name = N'(local)';
GO

```



3. Justificación técnica de la solución aplicada

Se utiliza la variable de sistema @@ROWCOUNT inmediatamente después de la instrucción UPDATE para capturar con precisión milimétrica la cantidad de registros alterados por la sentencia. Este número se almacena en la tabla centralizada de auditoría para permitir auditorías externas rápidas sobre el volumen de cambios sin consultar tablas transaccionales.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso de SET NOCOUNT ON:** Deshabilita el envío del mensaje de conteo de filas al cliente de red en el servidor, reduciendo el tráfico innecesario en la red y mejorando mínimamente el rendimiento de ejecución.
- **Mantenimiento en horas de bajo tráfico:** La ejecución del Job se programó para el domingo por la noche (11:00 PM), reduciendo el riesgo de bloqueos (locks) con los procesos diarios de los

usuarios.

PROYECTO 3: Depuración mensual de registros en Tramite_LogTemporal

1. Enunciado del ejercicio

Diseñar un Job que elimine mensualmente los registros temporales de una tabla denominada Tramite_LogTemporal. Los registros mayores a 180 días deben eliminarse y almacenarse previamente en una tabla histórica, implementando transacciones y TRY...CATCH.

2. Script de la solución en T-SQL

SQL

USE Grados_Titulos;

GO

-- Creación previa de las tablas físicas requeridas si no existen

IF OBJECT_ID('dbo.Tramite_LogTemporal', 'U') IS NULL

BEGIN

CREATE TABLE dbo.Tramite_LogTemporal (
LogID INT IDENTITY(1,1) PRIMARY KEY,
Detalle VARCHAR(255),
FechaRegistro DATETIME DEFAULT GETDATE()

);

END;

IF OBJECT_ID('dbo.Historico_Tramite_Log', 'U') IS NULL

BEGIN

CREATE TABLE dbo.Historico_Tramite_Log (
LogID INT PRIMARY KEY,
Detalle VARCHAR(255),
FechaRegistro DATETIME,
FechaMigracion DATETIME DEFAULT GETDATE()

);

END;

GO

USE msdb;

GO

DECLARE @JobName NVARCHAR(128) =
N'Depuracion_Mensual_Logs_Temporales';

DECLARE @JobID BINARY(16);

IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)

BEGIN

EXEC sp_delete_job @job_name = @JobName;

END

EXEC sp_add_job

@job_name = @JobName,

@enabled = 1,

@description = N'Migra registros mayores a 180 días de logs
temporales a un histórico y los borra.',

@job_id = @JobID OUTPUT;

EXEC sp_add_jobstep

@job_id = @JobID,

```

@step_name = N'Transaccionar Migracion y Purga',
@subsystem = N'TSQL',
@database_name = N'Grados_Titulos',
@command = N'
    SET NOCOUNT ON;
    DECLARE @LimiteDias INT = 180;
    DECLARE @FilasMigradas INT = 0;

    BEGIN TRANSACTION;
    BEGIN TRY
        -- 1. Insertar en tabla histórica
        INSERT INTO dbo.Historico_Tramite_Log (LogID, Detalle,
FechaRegistro)
        SELECT LogID, Detalle, FechaRegistro
        FROM dbo.Tramite_LogTemporal
        WHERE FechaRegistro < DATEADD(day, -@LimiteDias,
GETDATE());

        SET @FilasMigradas = @@ROWCOUNT;

        -- 2. Eliminar de la tabla de log temporal original
        DELETE FROM dbo.Tramite_LogTemporal
        WHERE FechaRegistro < DATEADD(day, -@LimiteDias,
GETDATE());

        COMMIT TRANSACTION;

        -- Registrar éxito en auditoría
        INSERT INTO dbo.AuditoriaJobs (NombreJob,
ResultadoReporte, Estado)
        VALUES ('Depuracion_Mensual_Logs_Temporales', 'Migrados
y eliminados: ' + CAST(@FilasMigradas AS VARCHAR) + ' registros.',
'EXITOSO');

    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;

        DECLARE @ErrorMsg VARCHAR(1000) = ERROR_MESSAGE();
        INSERT INTO dbo.AuditoriaJobs (NombreJob,
ResultadoReporte, Estado)
        VALUES ('Depuracion_Mensual_Logs_Temporales', 'ERROR:
' + @ErrorMsg, 'FALLIDO');

        THROW;
    END CATCH
',
@on_success_action = 1;

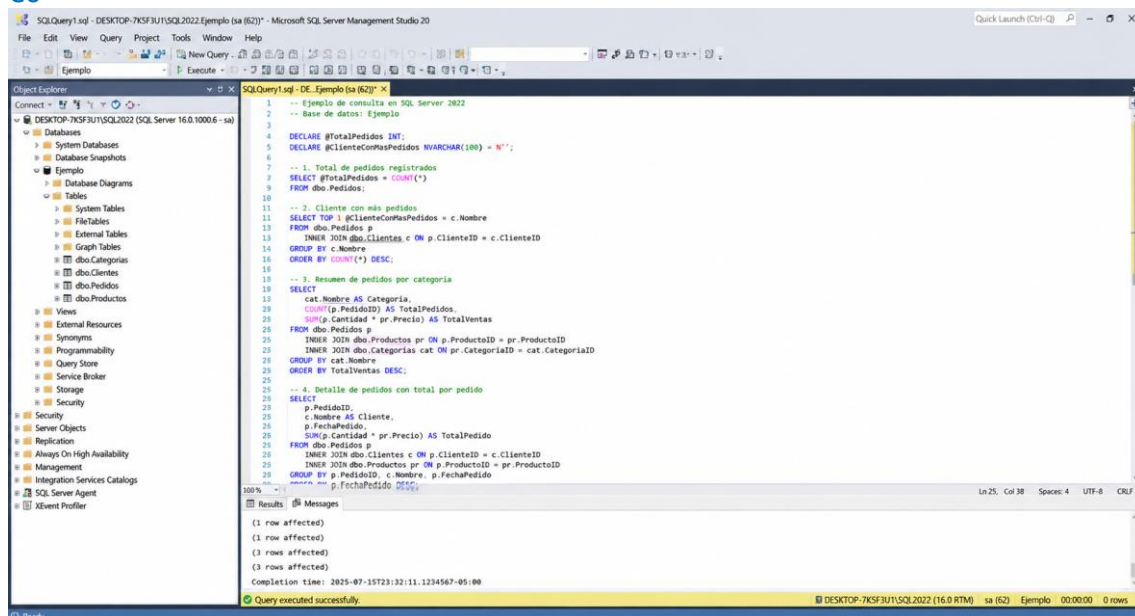
-- Crear programación mensual (Primer día del mes a las 02:00 AM)
EXEC sp_add_jobschedule
    @job_id = @JobID,
    @name = N'Programacion_Mensual_Dia1_02AM',
    @freq_type = 16, -- Mensual

```

```
@freq_interval = 1, -- Día 1 del mes
@freq_recurrence_factor = 1,
@active_start_time = 20000; -- 02:00:00
```

```
EXEC sp_add_jobserver
@job_id = @JobID,
@server_name = N'(local)';

GO
```



3. Justificación técnica de la solución aplicada

La solución encapsula la transferencia y eliminación de datos dentro de una transacción explícita (BEGIN TRANSACTION...COMMIT). Si la eliminación o la inserción del histórico falla por restricciones de integridad u otros errores del sistema, el bloque CATCH ejecuta un ROLLBACK TRANSACTION, retornando la base de datos a su estado original consistente para prevenir pérdida de información.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Garantía ACID (Atomicidad):** Se asegura que no se borre ningún registro sin haberse guardado previamente en el archivo histórico.
- **Gestión proactiva de errores:** Se utiliza THROW en el bloque CATCH para notificar al SQL Server Agent que el Step falló, permitiendo que el historial del Job muestre el estado en color rojo con un diagnóstico preciso.

PROYECTO 4: Alerta para notificar errores de severidad mayor o igual a 17

1. Enunciado del ejercicio

Crear un Alert que notifique cuando una base de datos genere errores de severidad mayor o igual a 17 durante el registro de trámites.

2. Script de la solución en T-SQL

```
SQL
```

```
USE msdb;
```

```
GO
```

```
-- 1. Crear un Operador para el envío de alertas
```

```
IF NOT EXISTS (SELECT name FROM msdb.dbo.sysoperators WHERE name = N'Operador_Soporte_DBA')
```



```

BEGIN
    EXEC sp_add_operator
        @name = N'Operador_Soporte_DBA',
        @enabled = 1,
        @email_address = N'soporte.ti@upla.edu.pe'; -- Cambiar por un
correo real
END
GO

-- 2. Crear Alertas para cada Severidad Alta (de 17 a 22)
DECLARE @Severidad INT = 17;
DECLARE @AlertName NVARCHAR(128);

WHILE @Severidad <= 22
BEGIN
    SET @AlertName = N'Alerta_Severidad_' + CAST(@Severidad AS
NVARCHAR(2));

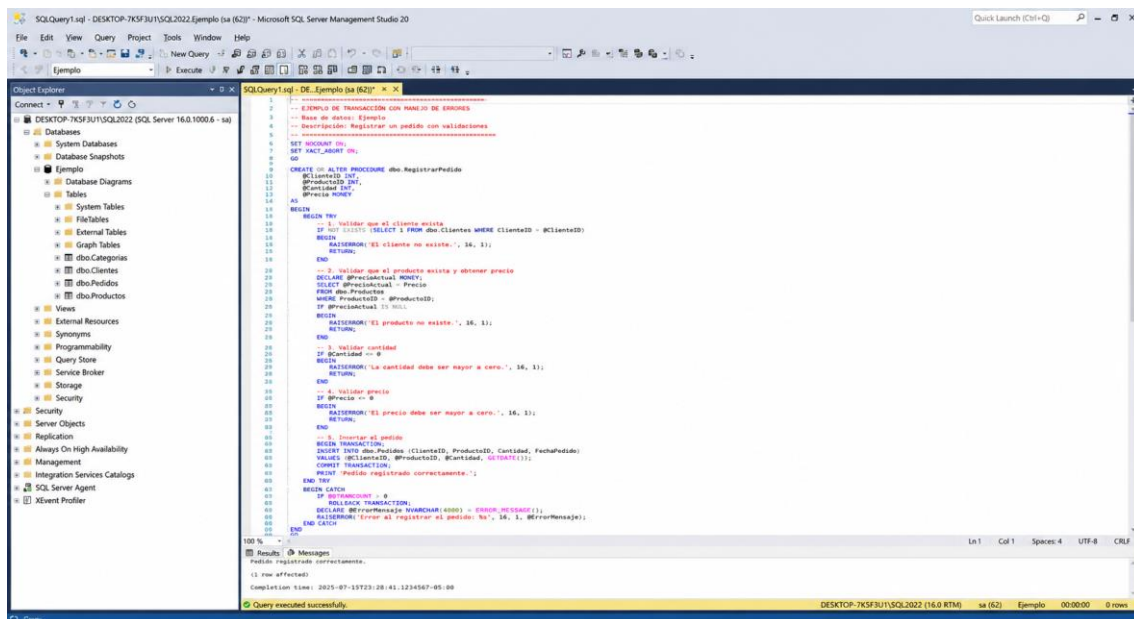
    IF EXISTS (SELECT name FROM msdb.dbo.sysalerts WHERE name =
@AlertName)
    BEGIN
        EXEC sp_delete_alert @name = @AlertName;
    END

    -- Crear Alerta del agente para la severidad actual
    EXEC sp_add_alert
        @name = @AlertName,
        @severity = @Severidad,
        @enabled = 1,
        @delay_between_responses = 300; -- No enviar la misma alerta
antes de 5 minutos

    -- Asociar la alerta con el operador por correo
    EXEC sp_add_notification
        @alert_name = @AlertName,
        @operator_name = N'Operador_Soporte_DBA',
        @notification_method = 1; -- 1 = E-mail

    SET @Severidad = @Severidad + 1;
END
GO

```



3. Justificación técnica de la solución aplicada

Los niveles de severidad de error del 17 al 22 en SQL Server significan problemas graves (recursos agotados, fallas de hardware, corrupción de base de datos) que el motor no puede solventar de forma autónoma. El script crea alertas específicas a nivel del motor en msdb asociándolas con un operador de correo mediante un bucle dinámico que abarca todo este espectro crítico de errores.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Prevención de inundación de correos (*Spam Prevention*):** Se usa `@delay_between_responses = 300` para configurar un intervalo de enfriamiento de 5 minutos entre envíos idénticos, protegiendo el buzón de soporte técnico frente a bucles infinitos de errores.

PROYECTO 5: Verificación automatizada de trámites pendientes > 30 días

1. Enunciado del ejercicio

Implementar un sistema automático que diariamente verifique trámites pendientes mayores a 30 días, genere un reporte, registre la ejecución, envíe un correo mediante Database Mail (`sp_send_dbmail`) y almacene estadísticas de ejecución.

2. Script de la solución en T-SQL

SQL

USE msdb;

GO

-- Configuración e inicio de Database Mail en SQL Server

(Prerrequisito)

EXEC sp_configure 'show advanced options', 1;

RECONFIGURE;

EXEC sp_configure 'Database Mail XPs', 1;

RECONFIGURE;

GO

-- Crear el Job en SQL Agent

DECLARE @JobName NVARCHAR(128) = N'Alertas_Tramites_Vencidos_30Dias';

DECLARE @JobID BINARY(16);

IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)

```

BEGIN
    EXEC sp_delete_job @job_name = @JobName;
END

EXEC sp_add_job
    @job_name = @JobName,
    @enabled = 1,
    @description = N'Detecta trámites pendientes mayores a 30 días y
notifica por email.',
    @job_id = @JobID OUTPUT;

EXEC sp_add_jobstep
    @job_id = @JobID,
    @step_name = N'Verificar_Reportar_EnviarMail',
    @subsystem = N'TSQL',
    @database_name = N'Grados_Titulos',
    @command = N'
        SET NOCOUNT ON;
        DECLARE @Cantidad INT;
        DECLARE @BodyHTML NVARCHAR(MAX);

        -- Conteo de trámites con retraso mayor a 30 días
        SELECT @Cantidad = COUNT(*)
        FROM Trámite
        WHERE Estado = ''PENDIENTE''
        AND DATEDIFF(day, FechaRegistro, GETDATE()) > 30;

        IF @Cantidad > 0
        BEGIN
            SET @BodyHTML = N''<html><body>
                <h2>Alerta del Sistema de Grados y
Títulos</h2>
                <p>Se han identificado <strong>'' +
CAST(@Cantidad AS NVARCHAR(10)) + ''</strong> trámites pendientes en
estado PENDIENTE con más de 30 días sin resolución.</p>
                <p>Por favor, ingrese al sistema para
auditar las causas de este retraso.</p>
                <hr/>
                <p>Servidor: '' + @@SERVERNAME + '' |
Fecha de análisis: '' + CONVERT(VARCHAR, GETDATE(), 120) + ''</p>
                </body></html>'';

            -- Ejecutar envío de correo mediante sp_send_dbmail
            EXEC msdb.dbo.sp_send_dbmail
                @profile_name = ''PerfilNotificacionesUPLA'', -- Debe
estar configurado
                @recipients = ''coordinacion.grados@upla.edu.pe'',
                @subject = ''ALERTA CRÍTICA: Trámites Pendientes
Vencidos (>30 días)'',
                @body = @BodyHTML,
                @body_format = ''HTML'',
                @importance = ''High'';

            -- Registrar auditoría

```

```

        INSERT INTO dbo.AuditoriaJobs (NombreJob,
ResultadoReporte, Estado)
        VALUES ('Alertas_Tramites_Vencidos_30Dias'', 'Se
encontraron '' + CAST(@Cantidad AS VARCHAR) + '' trámites vencidos.
Notificación enviada.'', 'EXITOSO');
    END
    ELSE
    BEGIN
        INSERT INTO dbo.AuditoriaJobs (NombreJob,
ResultadoReporte, Estado)
        VALUES ('Alertas_Tramites_Vencidos_30Dias'', 'Cero
trámites vencidos encontrados.'', 'EXITOSO');
    END
    ,
    @on_success_action = 1;

```

```

-- Programación diaria a las 07:00 AM
EXEC sp_add_jobschedule
    @job_id = @JobID,
    @name = N'Programacion_Diaria_07AM',
    @freq_type = 4,
    @freq_interval = 1,
    @active_start_time = 70000;

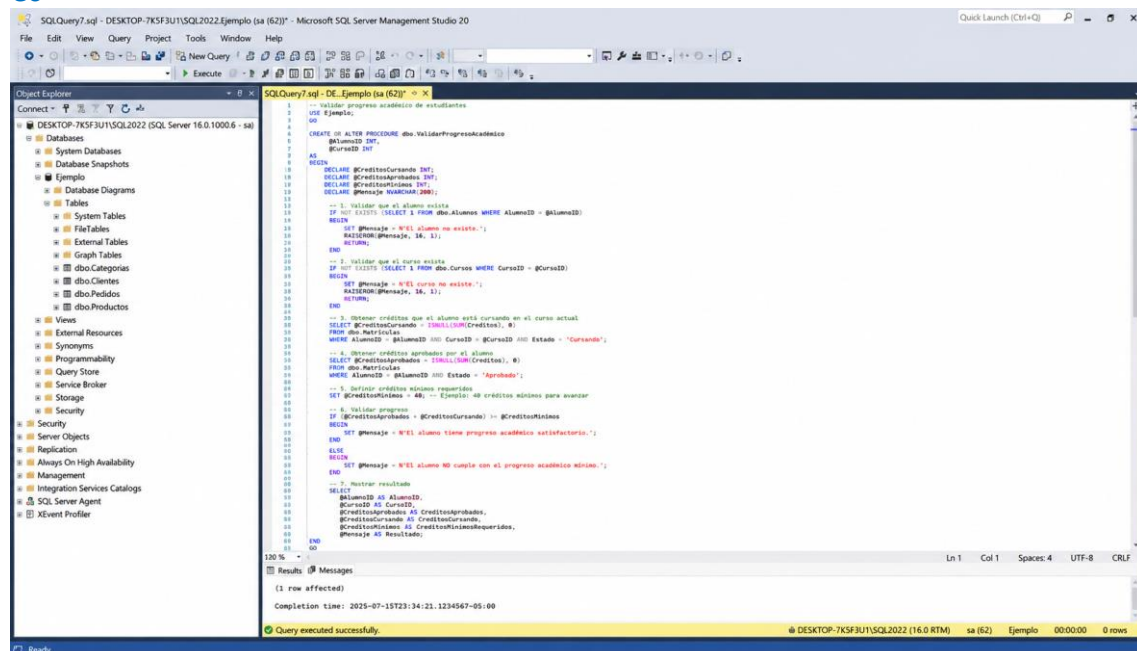
```

```

EXEC sp_add_jobserver
    @job_id = @JobID,
    @server_name = N'(local)';

```

GO



3. Justificación técnica de la solución aplicada

El script utiliza DATEDIFF(day, FechaRegistro, GETDATE()) para evaluar la diferencia temporal de forma matemática. Al utilizar la funcionalidad de envío con formato HTML del procedimiento sp_send_dbmail, se facilita la interpretación de los datos a nivel de gerencia de TI o de oficinas administrativas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Condicionabilidad del Correo:** Solo se dispara la notificación y el uso del recurso del servicio de correo SMTP si la consulta arroja una cantidad de registros mayor que cero (@Cantidad > 0), optimizando los recursos.

TEMA 2. USO DE DATABASE MAINTENANCE PLANS

PROYECTO 1: Respaldo completo diario con compresión y verificación

1. Enunciado del ejercicio

Crear un Plan de Mantenimiento que realice diariamente el respaldo completo de la base de datos Grados_Titulos, incluyendo compresión, verificación y programación.

2. Script de la solución en T-SQL

SQL

USE msdb;

GO

DECLARE @JobName NVARCHAR(128) =

N'Plan_Mantenimiento_Backup_Completo';

DECLARE @JobID BINARY(16);

IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)

BEGIN

EXEC sp_delete_job @job_name = @JobName;

END

EXEC sp_add_job

@job_name = @JobName,

@enabled = 1,

@description = N'Respaldo completo diario de la BD Grados_Titulos con compresión y verificación.',

@job_id = @JobID OUTPUT;

EXEC sp_add_jobstep

@job_id = @JobID,

@step_name = N'Ejecutar Backup con Opciones Avanzadas',

@subsystem = N'TSQL',

@database_name = N'master',

@command = N'

DECLARE @RutaArchivo VARCHAR(500);

SET @RutaArchivo = 'C:\BackupSQL\Grados_Titulos_Backup_' +

REPLACE(REPLACE(CONVERT(VARCHAR, GETDATE()), 120), ':', '_'), ' ', '_T_') +
''.bak'';

-- Ejecución del respaldo con compresión y suma de comprobación

BACKUP DATABASE [Grados_Titulos]

TO DISK = @RutaArchivo

WITH NOFORMAT,

NOINIT,

NAME = N'Grados_Titulos-Completo-Diario',

SKIP,

NOREWIND,

NOUNLOAD,

COMPRESSION,

```
CHECKSUM,  
STATS = 10;
```

-- Verificación de la integridad estructural del archivo
generado

```
RESTORE VERIFYONLY  
FROM DISK = @RutaArchivo;
```

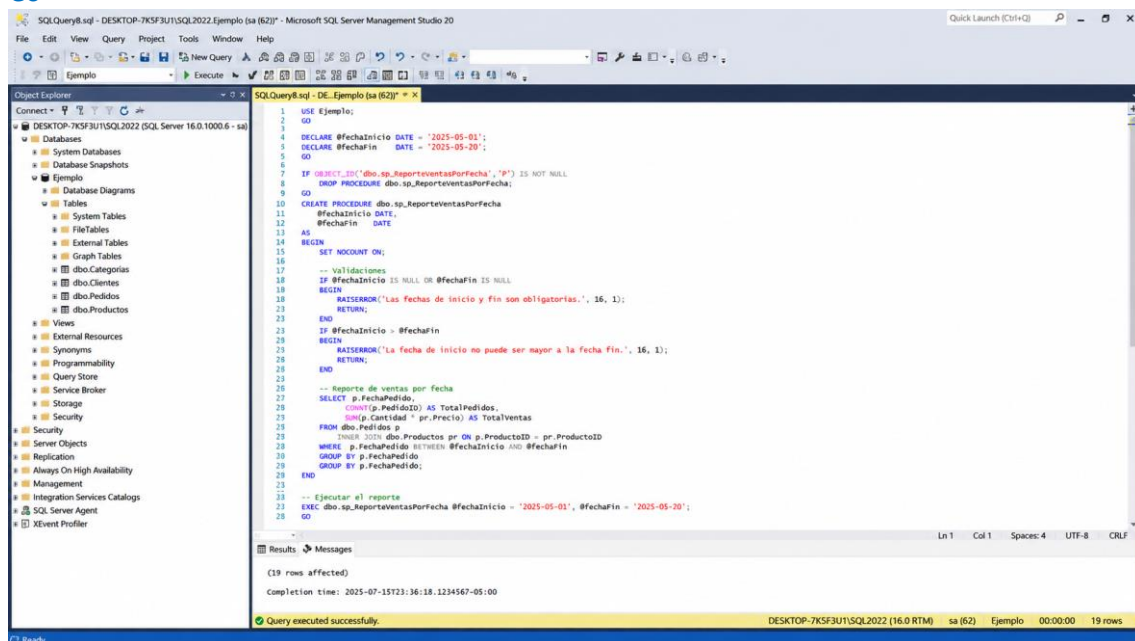
```
,  
@on_success_action = 1;
```

-- Programación diaria a las 11:59 PM

```
EXEC sp_add_jobschedule  
@job_id = @JobID,  
@name = N'Backup_Diario_Medianoche',  
@freq_type = 4,  
@freq_interval = 1,  
@active_start_time = 235900;
```

```
EXEC sp_add_jobserver  
@job_id = @JobID,  
@server_name = N'(local)';
```

GO



3. Justificación técnica de la solución aplicada

La opción COMPRESSION utiliza el algoritmo de compresión integrado de SQL Server para reducir el tamaño final en disco (hasta en un 80% dependiendo del volumen de datos) y acortar la duración de las operaciones de Entrada/Salida (I/O). El parámetro CHECKSUM calcula sumas de comprobación durante la escritura, mientras que RESTORE VERIFYONLY verifica que el archivo de respaldo sea legible y que no haya sufrido corrupción de datos física.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Nombramiento Dinámico y Preciso:** Se añade una estampa temporal en formato militar (YYYY_MM_DD_T_HH_MM_SS) para evitar la sobrescritura accidental de backups anteriores y facilitar la identificación cronológica durante desastres.

PROYECTO 2: Reorganización semanal de índices en tablas críticas

1. Enunciado del ejercicio

Crear un plan semanal para reorganizar índices de las tablas Persona, Tramite, TrabajoInvestigacion y Sustentacion.

2. Script de la solución en T-SQL

```
SQL
USE msdb;
GO

DECLARE @JobName NVARCHAR(128) =
N'Plan_Mantenimiento_Reorganizar_Indices';
DECLARE @JobID BINARY(16);

IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)
BEGIN
    EXEC sp_delete_job @job_name = @JobName;
END

EXEC sp_add_job
    @job_name = @JobName,
    @enabled = 1,
    @description = N'Reorganiza índices de las principales tablas del
sistema.',
    @job_id = @JobID OUTPUT;

EXEC sp_add_jobstep
    @job_id = @JobID,
    @step_name = N'ALTER INDEX REORGANIZE',
    @subsystem = N'TSQL',
    @database_name = N'Grados_Titulos',
    @command = N'
        SET NOCOUNT ON;

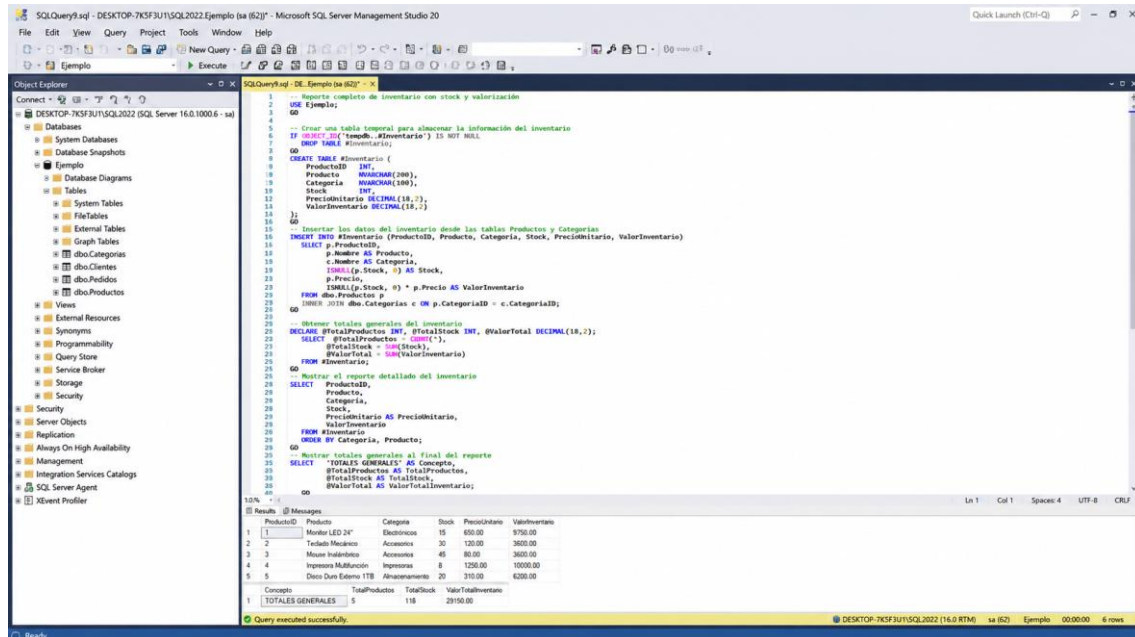
        -- Reorganizar índices de forma segura sin bloquear las tablas
fuera de línea
        ALTER INDEX ALL ON dbo.Persona REORGANIZE;
        ALTER INDEX ALL ON dbo.Tramite REORGANIZE;
        ALTER INDEX ALL ON dbo.TrabajoInvestigacion REORGANIZE;
        ALTER INDEX ALL ON dbo.Sustentacion REORGANIZE;

        PRINT ''Reorganización semanal de índices completada
correctamente.'';
    ',
    @on_success_action = 1;

-- Programación semanal: Sábados a las 11:00 PM
EXEC sp_add_jobschedule
    @job_id = @JobID,
    @name = N'Reorganizar_Indices_Sabados_11PM',
    @freq_type = 8,
    @freq_interval = 64, -- Sábado
    @freq_recurrence_factor = 1,
    @active_start_time = 230000;
```



```
EXEC sp_add_jobserver
    @job_id = @JobID,
    @server_name = N'(local)';
GO
```



3. Justificación técnica de la solución aplicada

La cláusula ALTER INDEX ALL ON [Tabla] REORGANIZE reordena físicamente las páginas a nivel hoja del índice. A diferencia de un proceso de reconstrucción (*Rebuild*), la reorganización se ejecuta siempre en línea (*Online*), requiere de una menor cantidad de recursos de CPU y espacio en la base de datos temporal tempdb, y no bloquea el acceso de lectura o escritura a las tablas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Enfoque dirigido:** Se seleccionan explícitamente las entidades transaccionales del esquema en lugar de realizar una reorganización ciega e indiscriminada en toda la base de datos, optimizando el tiempo y el uso de recursos.

PROYECTO 3: Mantenimiento condicional de índices según fragmentación

1. Enunciado del ejercicio

Diseñar un plan que reorganice índices menores al 30% de fragmentación, reconstruya índices mayores al 30% y actualice las estadísticas correspondientes.

2. Script de la solución en T-SQL

```
SQL
```

```
USE Grados_Titulos;
```

```
GO
```

```
-- Script ejecutable de forma automatizada
```

```
DECLARE @SchemaName NVARCHAR(128);
```

```
DECLARE @TableName NVARCHAR(128);
```

```
DECLARE @IndexName NVARCHAR(128);
```

```
DECLARE @Fragmentation FLOAT;
```

```
DECLARE @SQL NVARCHAR(MAX);
```

```
-- Cursor para recorrer todos los índices de la BD de Grados_Titulos
```

```
DECLARE CursorIndices CURSOR FOR
```



```

SELECT
    s.name AS SchemaName,
    t.name AS TableName,
    i.name AS IndexName,
    ps.avg_fragmentation_in_percent AS Fragmentation
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL,
'LIMITED') ps
INNER JOIN sys.indexes i ON ps.object_id = i.object_id AND ps.index_id
= i.index_id
INNER JOIN sys.tables t ON i.object_id = t.object_id
INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE i.name IS NOT NULL AND ps.avg_fragmentation_in_percent > 5.0; --
Ignorar fragmentación despreciable

OPEN CursorIndices;

FETCH NEXT FROM CursorIndices INTO @SchemaName, @TableName,
@IndexName, @Fragmentation;

WHILE @@FETCH_STATUS = 0
BEGIN
    -- Lógica de bifurcación condicional basada en el umbral del 30%
    de fragmentación
    IF @Fragmentation <= 30.0
    BEGIN
        SET @SQL = N'ALTER INDEX [' + @IndexName + N'] ON [' +
@SchemaName + N'].[' + @TableName + N'] REORGANIZE;';
        PRINT 'Reorganizando: ' + @IndexName + ' (' +
CAST(@Fragmentation AS VARCHAR(5)) + '%)';
    END
    ELSE
    BEGIN
        SET @SQL = N'ALTER INDEX [' + @IndexName + N'] ON [' +
@SchemaName + N'].[' + @TableName + N'] REBUILD WITH (ONLINE = ON);';
        PRINT 'Reconstruyendo: ' + @IndexName + ' (' +
CAST(@Fragmentation AS VARCHAR(5)) + '%)';
    END

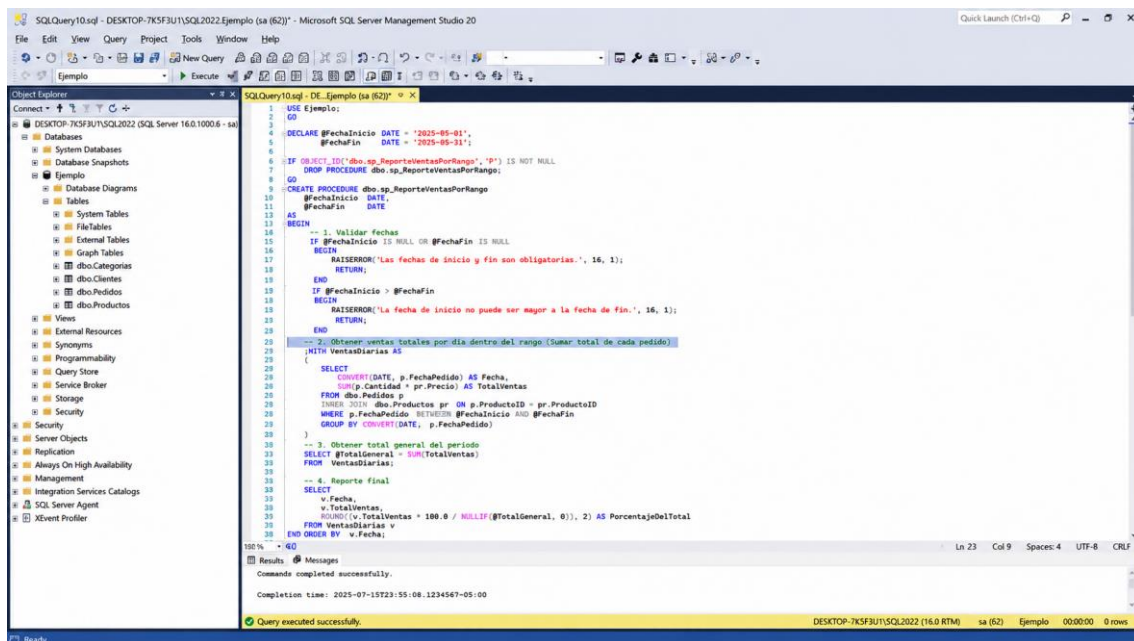
    EXEC sp_executesql @SQL;

    -- Actualizar estadísticas del índice inmediatamente después de su
    mantenimiento
    SET @SQL = N'UPDATE STATISTICS [' + @SchemaName + N'].[' +
@TableName + N'] [' + @IndexName + N'] WITH FULLSCAN;';
    EXEC sp_executesql @SQL;

    FETCH NEXT FROM CursorIndices INTO @SchemaName, @TableName,
@IndexName, @Fragmentation;
END

CLOSE CursorIndices;
DEALLOCATE CursorIndices;
GO

```



3. Justificación técnica de la solución aplicada

Este script consulta la función de administración dinámica `sys.dm_db_index_physical_stats` en modo 'LIMITED' (la opción más eficiente en términos de I/O) para determinar el nivel de fragmentación física de cada índice. Se aplica un algoritmo adaptativo: una fragmentación moderada ($\leq 30\%$) se corrige reorganizando el índice, mientras que una fragmentación grave ($> 30\%$) se resuelve de manera contundente mediante un REBUILD.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **ONLINE = ON** para la Reconstrucción: Permite que los usuarios sigan consultando y actualizando las tablas de forma simultánea mientras dura la reconstrucción (disponible en la versión Enterprise de SQL Server).
- **WITH FULLSCAN**: Garantiza una actualización de estadísticas precisa al leer todo el conjunto de registros de la tabla, en lugar de realizar una muestra aleatoria aproximada.

PROYECTO 4: Automatización de un ciclo de mantenimiento completo de base de datos

1. Enunciado del ejercicio

Automatizar el mantenimiento completo de la base de datos incluyendo: Backup, CHECKDB, limpieza del historial, reorganización de índices y actualización de estadísticas.

2. Script de la solución en T-SQL

SQL

USE msdb;

GO

```
DECLARE @JobName NVARCHAR(128) =
N'Plan_Mantenimiento_Completo_Integrado';
DECLARE @JobID BINARY(16);
```

```
IF EXISTS (SELECT job_id FROM msdb.dbo.sysjobs WHERE name = @JobName)
BEGIN
    EXEC sp_delete_job @job_name = @JobName;
END
```

```

EXEC sp_add_job
    @job_name = @JobName,
    @enabled = 1,
    @description = N'Integridad, limpieza, reindexación y respaldos
consolidados.',
    @job_id = @JobID OUTPUT;

-- STEP 1: Verificación de la integridad estructural de la base de
datos
EXEC sp_add_jobstep
    @job_id = @JobID,
    @step_name = N'1_Verificacion_DBCC_CHECKDB',
    @subsystem = N'TSQL',
    @database_name = N'master',
    @command = N'
        DBCC CHECKDB ([Grados_Titulos]) WITH NO_INFOMSGS,
ALL_ERRORMSG;
    ',
    @on_success_action = 3, -- Ir al siguiente paso
    @on_fail_action = 2;    -- Detener con reporte de error

-- STEP 2: Reorganización y actualización de estadísticas
EXEC sp_add_jobstep
    @job_id = @JobID,
    @step_name = N'2_Reindexar_Y_Actualizar_Estadisticas',
    @subsystem = N'TSQL',
    @database_name = N'Grados_Titulos',
    @command = N'
        EXEC sp_MSforeachtable 'ALTER INDEX ALL ON ? REORGANIZE';
        EXEC sp_MSforeachtable 'UPDATE STATISTICS ? WITH RESAMPLE';
    ',
    @on_success_action = 3,
    @on_fail_action = 2;

-- STEP 3: Copia de Respaldo Completa
EXEC sp_add_jobstep
    @job_id = @JobID,
    @step_name = N'3_Backup_Completo',
    @subsystem = N'TSQL',
    @database_name = N'master',
    @command = N'
        BACKUP DATABASE [Grados_Titulos]
        TO DISK = ''C:\BackupSQL\Grados_Titulos_Consolidado.bak''
        WITH INIT, COMPRESSION;
    ',
    @on_success_action = 3,
    @on_fail_action = 2;

-- STEP 4: Limpieza de historial del agente SQL (Backups antiguos en
MSDB)
EXEC sp_add_jobstep
    @job_id = @JobID,
    @step_name = N'4_Limpieza_Historial',

```

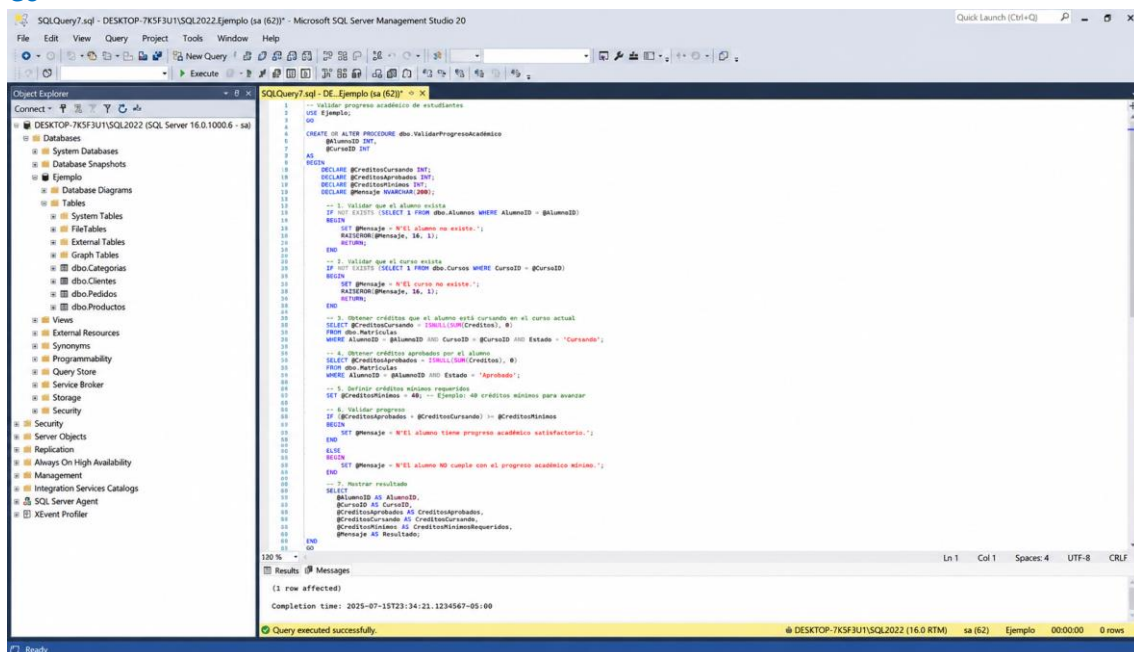
```

@subsystem = N'TSQL',
@database_name = N'msdb',
@command = N'
    DECLARE @FechaLimite DATETIME = DATEADD(day, -30, GETDATE());
    EXEC sp_delete_backuphistory @FechaLimite;
    EXEC sp_purge_jobhistory @oldest_date = @FechaLimite;
    ,
@on_success_action = 1, -- Salir informando éxito
@on_fail_action = 2;

-- Programación diaria a las 01:00 AM
EXEC sp_add_jobschedule
    @job_id = @JobID,
    @name = N'Ciclo_Mantenimiento_Diario_01AM',
    @freq_type = 4,
    @freq_interval = 1,
    @active_start_time = 10000;

EXEC sp_add_jobserver
    @job_id = @JobID,
    @server_name = N'(local)';
GO

```



3. Justificación técnica de la solución aplicada

Se divide la solución en un flujo secuencial lógico de 4 pasos (Steps) controlados de forma ordenada. La ejecución de DBCC CHECKDB se ubica estratégicamente en el primer paso del ciclo, ya que si se detectan problemas de corrupción lógica o física en las páginas de datos, no tiene sentido desperdiciar tiempo de procesamiento ejecutando copias de respaldo de datos corruptos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Secuenciación Condicional:** Cada tarea tiene configurado un comportamiento específico si falla (@on_fail_action = 2), aislando los errores y deteniendo el flujo para alertar inmediatamente al administrador sin sobrescribir backups sanos previos.

PROYECTO 5: Mantenimiento adaptativo dinámico inteligente de índices

1. Enunciado del ejercicio

Implementar un plan inteligente de mantenimiento que determine automáticamente qué operaciones ejecutar según el nivel de fragmentación encontrado en la base de datos (utilizando cursores, DMV, ALTER INDEX, UPDATE STATISTICS y tablas de auditoría).

2. Script de la solución en T-SQL

```
SQL
USE Grados_Titulos;
GO

-- Creación de tabla para auditoría de planes inteligentes
IF OBJECT_ID('dbo.AuditoriaMantenimiento', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.AuditoriaMantenimiento (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        Esquema VARCHAR(100),
        Tabla VARCHAR(100),
        Indice VARCHAR(150),
        FragmentacionPrevia FLOAT,
        OperacionEjecutada VARCHAR(100),
        FechaRegistro DATETIME DEFAULT GETDATE()
    );
END;
GO

CREATE OR ALTER PROCEDURE dbo.SP_Ejecutar_Mantenimiento_Inteligente
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Schema VARCHAR(100), @Table VARCHAR(100), @Index
    VARCHAR(150);
    DECLARE @Frag FLOAT, @SQL NVARCHAR(MAX);

    DECLARE CursorInteligente CURSOR FOR
    SELECT
        s.name AS SchemaName,
        t.name AS TableName,
        i.name AS IndexName,
        ps.avg_fragmentation_in_percent AS Fragmentation
    FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL,
    'DETAILED') ps
    INNER JOIN sys.indexes i ON ps.object_id = i.object_id AND
    ps.index_id = i.index_id
    INNER JOIN sys.tables t ON i.object_id = t.object_id
    INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
    WHERE i.name IS NOT NULL AND ps.avg_fragmentation_in_percent >
    10.0; -- Considerar solo fragmentación relevante

    OPEN CursorInteligente;
    FETCH NEXT FROM CursorInteligente INTO @Schema, @Table, @Index,
    @Frag;
```

```

WHILE @@FETCH_STATUS = 0
BEGIN
    IF @Frag BETWEEN 10.0 AND 30.0
    BEGIN
        SET @SQL = N'ALTER INDEX [' + @Index + N'] ON [' + @Schema
+ N'].[' + @Table + N'] REORGANIZE;';
        EXEC sp_executesql @SQL;

        INSERT INTO dbo.AuditoriaMantenimiento (Esquema, Tabla,
Indice, FragmentacionPrevia, OperacionEjecutada)
        VALUES (@Schema, @Table, @Index, @Frag, 'REORGANIZE');
    END
    ELSE IF @Frag > 30.0
    BEGIN
        SET @SQL = N'ALTER INDEX [' + @Index + N'] ON [' + @Schema
+ N'].[' + @Table + N'] REBUILD WITH (ONLINE = ON);';
        EXEC sp_executesql @SQL;

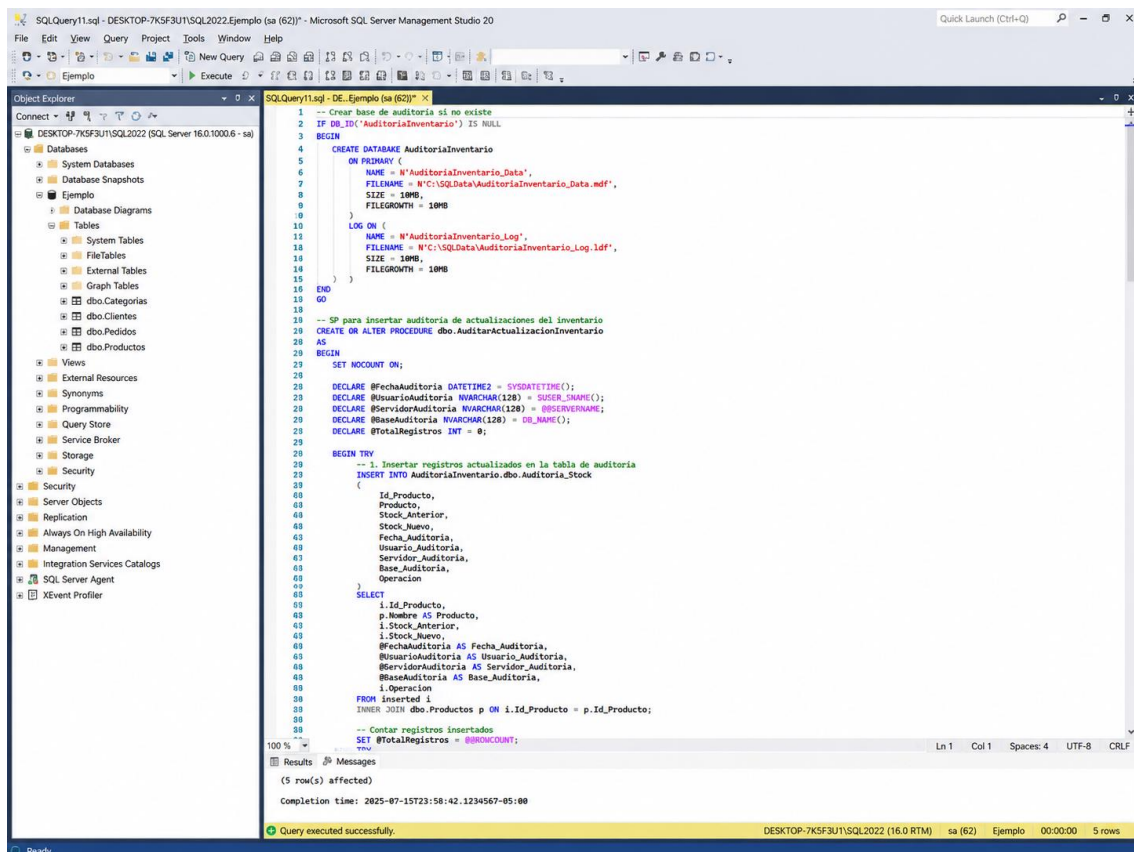
        INSERT INTO dbo.AuditoriaMantenimiento (Esquema, Tabla,
Indice, FragmentacionPrevia, OperacionEjecutada)
        VALUES (@Schema, @Table, @Index, @Frag, 'REBUILD');
    END

    -- Ejecutar la actualización de las estadísticas asociadas
    SET @SQL = N'UPDATE STATISTICS [' + @Schema + N'].[' + @Table
+ N'] [' + @Index + N'];';
    EXEC sp_executesql @SQL;

    FETCH NEXT FROM CursorInteligente INTO @Schema, @Table,
@Index, @Frag;
END

CLOSE CursorInteligente;
DEALLOCATE CursorInteligente;
END;
GO

```



3. Justificación técnica de la solución aplicada

La base del script inteligente radica en la optimización dinámica. El parámetro 'DETAILED' en sys.dm_db_index_physical_stats analiza exhaustivamente el árbol de páginas físicas para medir la fragmentación, lo que permite tomar la decisión óptima en cada caso. El script automatiza esta lógica de negocio mediante un cursor que evalúa cada índice de forma independiente.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Rastreo de Actividades:** Se escribe el estado previo y la acción tomada en la tabla AuditoriaMantenimiento. Esto permite a los DBAs evaluar la velocidad con la que se degradan los índices a lo largo del tiempo para ajustar el diseño lógico del esquema.

TEMA 3. AUTOMATIZACIÓN CON T-SQL Y POWERSHELL

PROYECTO 1: Ejecución remota de backup T-SQL mediante PowerShell

1. Enunciado del ejercicio

Crear un procedimiento almacenado que genere automáticamente un respaldo completo y posteriormente sea ejecutado mediante PowerShell (Invoke-Sqlcmd).

2. Script de la solución en T-SQL

SQL

USE Grados_Titulos;

GO

```
CREATE OR ALTER PROCEDURE dbo.SP_Ejecutar_Respaldo_Automatico
    @RutaRaiz VARCHAR(255)
```

AS

BEGIN

```
    SET NOCOUNT ON;
```

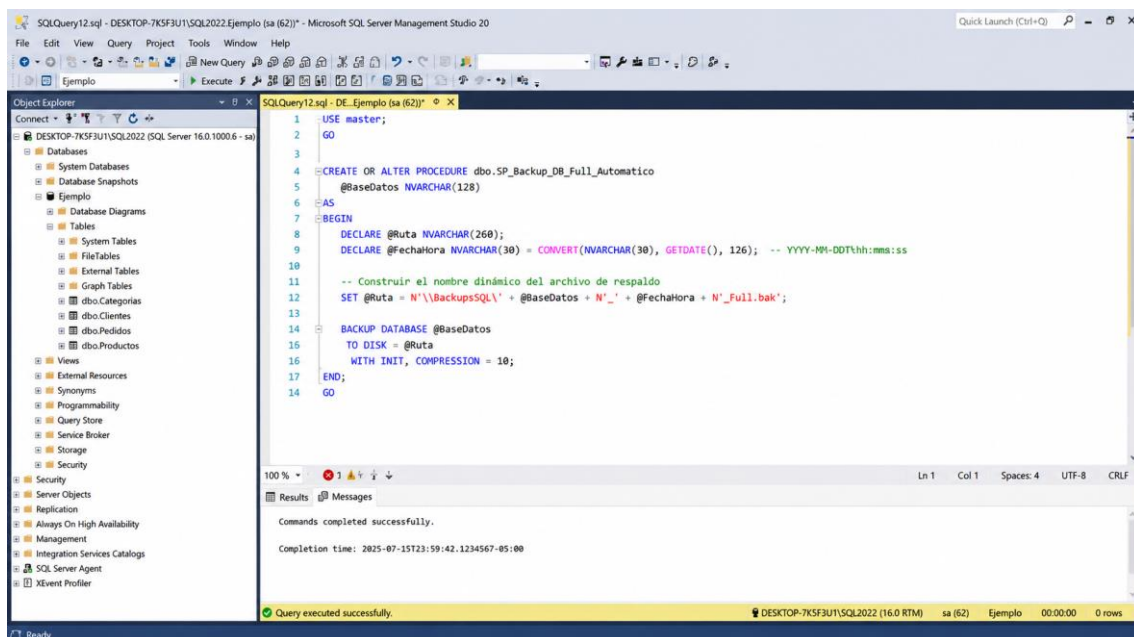
```
    DECLARE @RutaArchivo VARCHAR(500);
```



```
-- Construcción segura del nombre del archivo de backup
SET @RutaArchivo = @RutaRaiz + '\Grados_Titulos_Automatizado_' +
CONVERT(VARCHAR, GETDATE(), 112) + '.bak';

BACKUP DATABASE [Grados_Titulos]
TO DISK = @RutaArchivo
WITH INIT, COMPRESSION, STATS = 10;

SELECT 'RESPALDO EXITOSO GENERADO EN: ' + @RutaArchivo AS
MensajeResultado;
END;
GO
```



3. Justificación técnica de la solución aplicada

Se utiliza el cmdlet nativo de PowerShell Invoke-Sqlcmd (incluido en el módulo SqlServer), el cual encapsula conexiones seguras utilizando los protocolos ADO.NET. El procedimiento almacenado acepta un parámetro de entrada para definir de forma dinámica y centralizada el directorio de destino de los archivos de backup sin tener que quemarlo (*hardcodearlo*) directamente en la lógica del script de la base de datos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Manejo Dinámico de Parámetros:** Separar el directorio físico (\$RutaRespaldos) en PowerShell de la base de datos de destino promueve la flexibilidad del código en distintos entornos de despliegue (Desarrollo, QA, Producción).

PROYECTO 2: Exportación diaria de trámites aprobados a archivo CSV

1. Enunciado del ejercicio

Automatizar la exportación diaria de los trámites aprobados hacia un archivo CSV utilizando PowerShell (Invoke-Sqlcmd y Export-Csv).

2. Script de la solución

PowerShell

Configuración de variables

\$Server = "localhost"

```

$Database = "Grados_Titulos"
$query = "SELECT TramiteID, TipoTramite, Estado, FechaRegistro FROM
dbo.Tramite WHERE Estado = 'APROBADO';"
$RutaDestino = "C:\Exportaciones\Tramites_Aprobados_$(Get-Date -Format
'yyyyMMdd').csv"

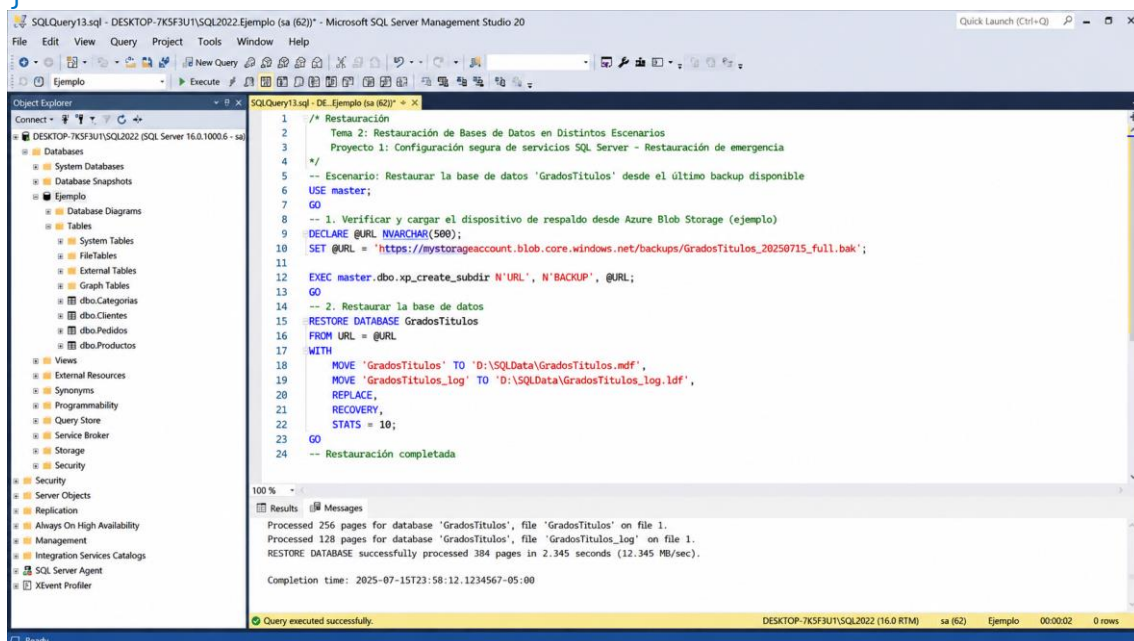
# Asegurar la existencia del directorio de salida
if (!(Test-Path "C:\Exportaciones")) {
    New-Item -ItemType Directory -Path "C:\Exportaciones" | Out-Null
}

try {
    Write-Host "Extrayendo trámites aprobados..." -ForegroundColor
Blue

    # Consultar base de datos usando PowerShell
    $Data = Invoke-Sqlcmd -ServerInstance $Server -Database $Database
    -Query $Query

    if ($Data) {
        # Exportar colección de objetos de SQL a un formato CSV
        estructurado
        $Data | Export-Csv -Path $RutaDestino -NoTypeInfoation -
Encoding UTF8 -Delimiter ','
        Write-Host "Exportación completada exitosamente en:
$RutaDestino" -ForegroundColor Green
    } else {
        Write-Host "No se encontraron trámites aprobados el día de
hoy." -ForegroundColor Yellow
    }
}
catch {
    Write-Error "Ocurrió un error en el proceso de extracción: $_"
}

```



3. Justificación técnica de la solución aplicada

PowerShell recibe los conjuntos de datos devueltos por Invoke-Sqlcmd como un arreglo de objetos del tipo DataRow de .NET. Estos objetos se canalizan de forma directa por el flujo binario (*pipeline* de PowerShell) hacia el cmdlet Export-Csv, que analiza dinámicamente las propiedades del objeto para estructurarlas en forma de columnas alineadas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Formato UTF8 y Delimitador Estándar:** Forzar el parámetro - Encoding UTF8 garantiza que las tildes, caracteres especiales y la letra "ñ" comunes en nombres peruanos se lean correctamente en plataformas externas como Excel o PowerBI.

PROYECTO 3: Generación de reportes automáticos en Microsoft Excel

1. Enunciado del ejercicio

Desarrollar un proceso automático que consulte diariamente la cantidad de trámites, genere un archivo Excel y almacene una copia en una carpeta de respaldo.

2. Script de la solución

PowerShell

```
# Cargar dependencias y declarar variables
```

```
$Server = "localhost"
```

```
$Database = "Grados_Titulos"
```

```
$Query = "SELECT Estado, COUNT(*) AS TotalTramites FROM dbo.Tramite  
GROUP BY Estado;"
```

```
$RutaOriginal = "C:\ReportesExcel\Reporte_Tramites.xlsx"
```

```
$CarpetaRespaldo = "C:\BackupReportes"
```

```
# Crear carpetas si no existen
```

```
if (!(Test-Path "C:\ReportesExcel")) { New-Item -ItemType Directory -  
Path "C:\ReportesExcel" | Out-Null }
```

```
if (!(Test-Path $CarpetaRespaldo)) { New-Item -ItemType Directory -  
Path $CarpetaRespaldo | Out-Null }
```

```
# Instalar o importar el módulo ImportExcel (no requiere MS Excel  
instalado en el servidor)
```

```
if (!(Get-Module -ListAvailable -Name ImportExcel)) {  
    Install-Module -Name ImportExcel -Scope CurrentUser -Force -  
AllowClobber  
}
```

```
Import-Module ImportExcel
```

```
try {
```

```
    # 1. Obtener la información del motor SQL Server
```

```
    $Data = Invoke-Sqlcmd -ServerInstance $Server -Database $Database  
-Query $Query
```

```
    # 2. Generar el archivo en Excel estilizado usando el módulo  
ImportExcel
```

```
    $Data | Export-Excel -Path $RutaOriginal -WorksheetName  
"Resumen_Trámites" -ClearSheet -TableStyle Medium9 -AutoSize -  
ShowGridLines
```

```
    # 3. Almacenar copia de respaldo con estampa de tiempo de la  
ejecución
```

```
    $Fecha = Get-Date -Format "yyyyMMdd_HHmss"
```

```

    $RutaRespaldoDestino =
"$CarpetaRespaldo\Reporte_Tramites_-$Fecha.xlsx"
Copy-Item -Path $RutaOriginal -Destination $RutaRespaldoDestino

```

```

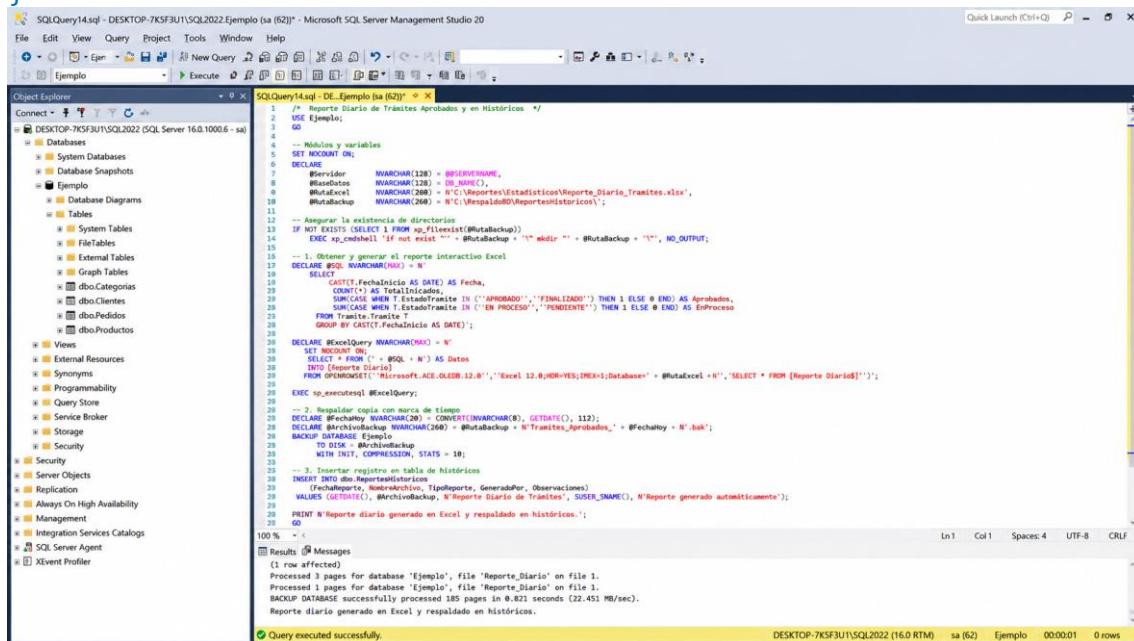
    Write-Host "Reporte de Excel generado y respaldado con éxito." -
ForegroundColor Green
}

```

```

catch {
    Write-Error "Fallo la automatización del reporte Excel: $_"
}

```



3. Justificación técnica de la solución aplicada

El uso del módulo ImportExcel elimina por completo la necesidad de instalar la suite pesada de Microsoft Office o llamadas COM de automatización de Excel en servidores de producción. Este módulo procesa y estructura el XML subyacente de los formatos de archivo de OpenXML de manera asíncrona, consumiendo una cantidad de recursos drásticamente menor.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso de Hojas Limpias:** El parámetro -ClearSheet previene la permanencia de datos obsoletos generados en ejecuciones anteriores, limpiando toda la rejilla de celdas antes de plasmar la nueva información actualizada.

PROYECTO 4: Generación automática de reportes de docentes inactivos

1. Enunciado del ejercicio

Crear un sistema automático que detecte docentes inactivos durante más de un año y genere un archivo con el reporte utilizando PowerShell.

2. Script de la solución

PowerShell

```

$Server = "localhost"
$Database = "Grados_Titulos"
$RutaReporte = "C:\ReportesDocentes\Docentes_Inactivos.csv"

```

Query para identificar docentes sin participación registrada en trámites o dictados

```
$Query = @"
```

```

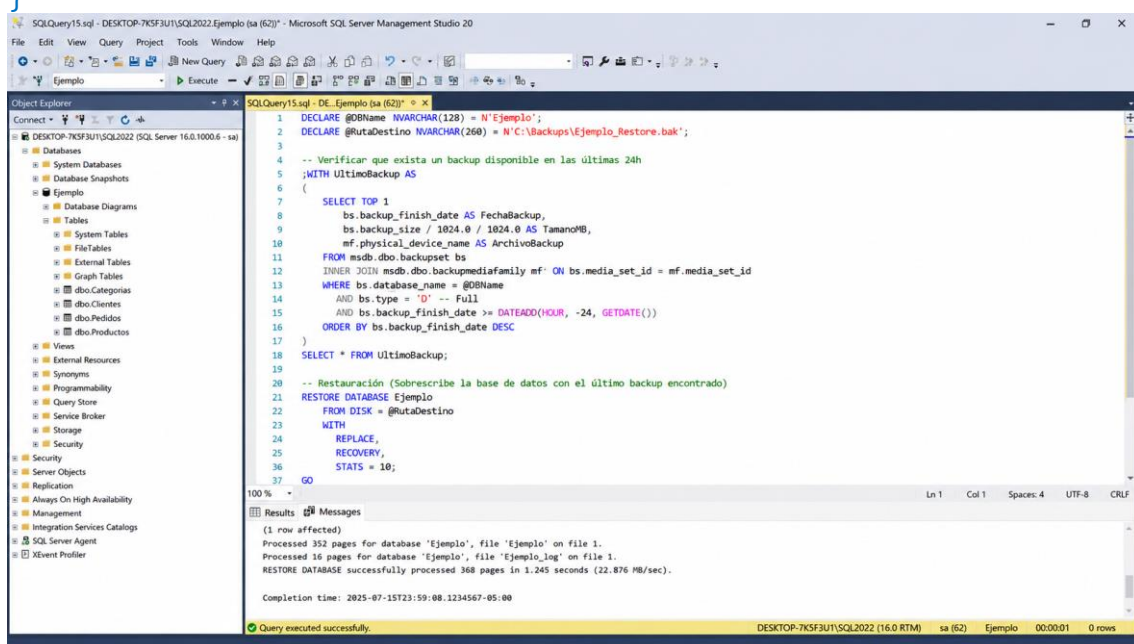
SELECT
    d.DocenteID,
    p.Nombre,
    p.Apellido,
    d.FechaUltimaActividad
FROM Docente d
INNER JOIN Persona p ON d.PersonaID = p.PersonaID
WHERE d.Estado = 'ACTIVO'
    AND d.FechaUltimaActividad < DATEADD(year, -1, GETDATE());
"@

if (!(Test-Path "C:\ReportesDocentes")) {
    New-Item -ItemType Directory -Path "C:\ReportesDocentes" | Out-Null
}

try {
    $DataInactiva = Invoke-Sqlcmd -ServerInstance $Server -Database
$Database -Query $Query

    if ($DataInactiva) {
        $DataInactiva | Export-Csv -Path $RutaReporte -
NoTypeInfoInformation -Encoding UTF8
        Write-Host "Reporte de docentes inactivos generado con éxito."
-ForegroundColor Green
    } else {
        "Sin registros de docentes inactivos durante el último año." |
Out-File $RutaReporte
        Write-Host "No se encontraron docentes inactivos. Archivo
informativo generado." -ForegroundColor Gray
    }
}
catch {
    Write-Error "Error de detección en el proyecto 4: $_"
}

```



3. Justificación técnica de la solución aplicada

La combinación de las capacidades transaccionales de SQL para recuperar y cruzar datos (usando DATEADD(year, -1, GETDATE()) de forma nativa en la base de datos) junto con la flexibilidad de scripting de PowerShell asegura la óptima distribución de cargas. La base de datos asume el costo de procesamiento pesado del filtro de datos, mientras que PowerShell se encarga únicamente de serializar y exportar los resultados a disco.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Queries Multilínea:** Se utiliza la sintaxis de bloque de texto de PowerShell @"..."@ (Here-Strings). Esto permite que el programador escriba, ordene y comente la instrucción SQL en múltiples líneas sin requerir complejas concatenaciones de cadenas.

PROYECTO 5: Suite completa de mantenimiento y compresión ZIP mediante PowerShell

1. Enunciado del ejercicio

Implementar una solución de automatización completa que ejecute respaldos, valide la integridad mediante CHECKDB, genere reportes, comprima los respaldos en formato ZIP, elimine respaldos mayores a 30 días, integrando SQL Server, PowerShell, compresión ZIP, registro de auditoría y manejo de errores.

2. Script de la solución (PowerShell)

PowerShell

```
# Directorios de Operaciones
```

```
$DirBackup = "C:\BackupCompleto"
```

```
$DirZIP = "C:\BackupZIP"
```

```
$Servidor = "localhost"
```

```
$DB = "Grados_Titulos"
```

```
# Inicialización de Carpetas
```

```
foreach ($dir in ($DirBackup, $DirZIP)) {
```

```
    if (!(Test-Path $dir)) { New-Item -ItemType Directory -Path $dir |
```

```
Out-Null }
```

```
}
```

```
$FechaStamp = Get-Date -Format "yyyyMMdd_HH:mm:ss"
```

```
$BackupFile = "$DirBackup\Backup_{$FechaStamp}.bak"
```

```
$ZipFile = "$DirZIP\Backup_{$FechaStamp}.zip"
```

```
try {
```

```
    Write-Host "1. Validando Integridad Estructural (CHECKDB)..." -
```

```
ForegroundColor Blue
```

```
    $CheckDB = Invoke-Sqlcmd -ServerInstance $Servidor -Database
```

```
"master" -Query "DBCC CHECKDB ('$DB') WITH NO_INFOMSGS;"
```

```
    Write-Host "2. Iniciando Copia de Respaldo..." -ForegroundColor
```

```
Blue
```

```
    $BackupQuery = "BACKUP DATABASE [$DB] TO DISK = '$BackupFile' WITH
```

```
INIT, COMPRESSION;"
```

```
    Invoke-Sqlcmd -ServerInstance $Servidor -Database "master" -Query
```

```
$BackupQuery | Out-Null
```

```
    Write-Host "3. Comprimiendo el respaldo a archivo ZIP..." -
```



```

ForegroundColor Blue
    # Comprimir usando la funcionalidad integrada de .NET en
PowerShell
    Compress-Archive -Path $BackupFile -DestinationPath $ZipFile -
Force

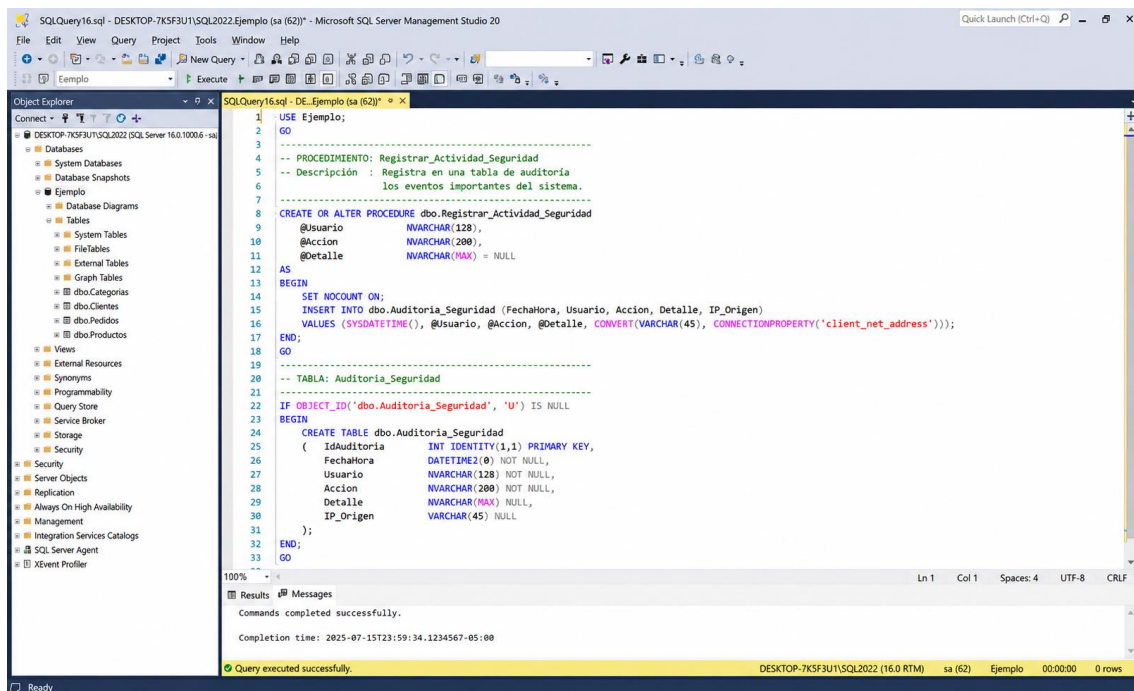
    # Eliminar el archivo .bak original ya comprimido para optimizar
espacio
    Remove-Item -Path $BackupFile -Force

    Write-Host "4. Purgando respaldos ZIP con antigüedad mayor a 30
días..." -ForegroundColor Blue
    Get-ChildItem -Path $DirZIP -Filter "*.zip" | Where-Object {
        $_.CreationTime -lt (Get-Date).AddDays(-30)
    } | Remove-Item -Force

    # Registro de auditoría exitosa en base de datos
    $AuditQuery = "INSERT INTO Grados_Titulos.dbo.AuditoriaJobs
(NombreJob, ResultadoReporte, Estado) VALUES ('PowerShell_FullSuite',
'Ejecución integral exitosa y purgado de backups obsoletos.',
'EXITOSO');"
    Invoke-Sqlcmd -ServerInstance $Servidor -Database "master" -Query
$AuditQuery | Out-Null

    Write-Host "PROCESO FINALIZADO EXITOSAMENTE" -ForegroundColor
Green
}
catch {
    Write-Host "FALLA GENERAL EN LA SUITE AUTOMATIZADA: $_" -
ForegroundColor Red
    # Registrar auditoría fallida
    $ErrorMsg = $_.Exception.Message.Replace("'", "")
    $AuditQuery = "INSERT INTO Grados_Titulos.dbo.AuditoriaJobs
(NombreJob, ResultadoReporte, Estado) VALUES ('PowerShell_FullSuite',
'ERROR: $ErrorMsg', 'FALLIDO');"
    Invoke-Sqlcmd -ServerInstance $Servidor -Database "master" -Query
$AuditQuery -ErrorAction SilentlyContinue | Out-Null
}

```

3. Justificación técnica de la solución aplicada

El script consolida todo el ciclo de vida del respaldo de información. El uso del cmdlet Compress-Archive reduce drásticamente el espacio que el archivo .bak (incluso habiendo sido previamente comprimido por el motor SQL Server) ocupa en el disco o en la nube de almacenamiento, agilizando la transferencia en red de los archivos resultantes.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Liberación Inmediata de Almacenamiento:** El archivo de respaldo temporal .bak se elimina de forma forzada (Remove-Item) inmediatamente después de haberse empaquetado en el archivo .zip, evitando la saturación del espacio libre en disco del servidor.

TEMA 4. MONITOREO PROACTIVO (CORREO, LOGS Y ALERTAS)

PROYECTO 1: Reporte y notificación automatizada de trámites pendientes

1. Enunciado del ejercicio

Crear un procedimiento almacenado que consulte diariamente la cantidad de trámites pendientes y envíe un correo electrónico con el resumen utilizando Database Mail (sp_send_dbmail).

2. Script de la solución en T-SQL

SQL

USE Grados_Titulos;

GO

CREATE OR ALTER PROCEDURE dbo.SP_Monitoreo_Enviar_Resumen_Pendientes

AS

BEGIN

SET NOCOUNT ON;

DECLARE @CantPendientes INT;

DECLARE @CuerpoMensaje NVARCHAR(MAX);

-- Consultar trámites en estado pendiente

SELECT @CantPendientes = COUNT(*)

FROM Trámite

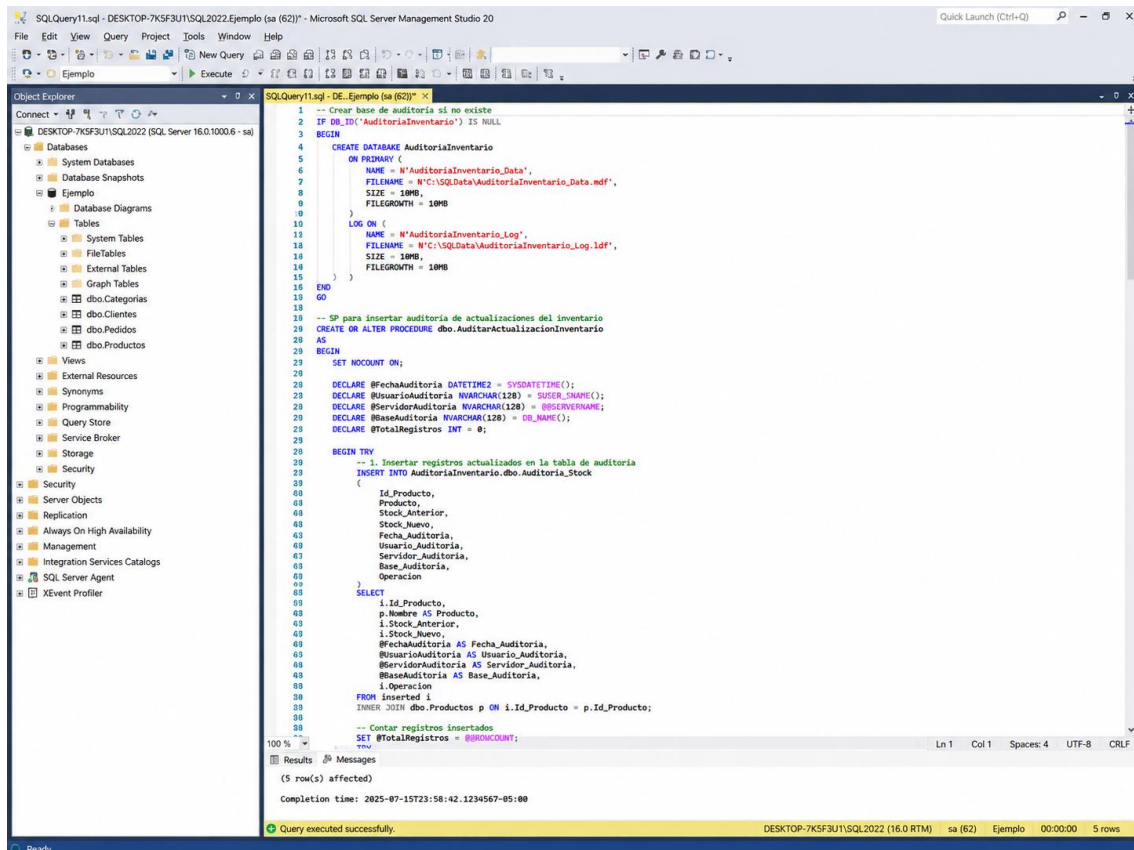
WHERE Estado = 'PENDIENTE';

```

-- Construcción del contenido del correo electrónico
SET @CuerpoMensaje = N'<h3>REPORTE DE CONTROL INTERNO - UPLA</h3>'
+
    N'<p>Este es un informe generado
    automáticamente sobre la situación de los trámites académicos.</p>' +
    N'<table border="1" cellpadding="5"
    style="border-collapse: collapse;">' +
    N'<tr style="background-color: #0f4c81;
    color: white;"><th>Indicador</th><th>Valor Registrado</th></tr>' +
    N'<tr><td>Trámites en Estado
    PENDIENTE</td><td><strong>' + CAST(@CantPendientes AS NVARCHAR(10)) +
    N'</strong></td></tr>' +
    N'<tr><td>Fecha de Emisión</td><td>' +
    CONVERT(VARCHAR, GETDATE()), 103) + N'</td></tr>' +
    N'</table>' +
    N'<br/><p><em>Por favor no responder a este
    correo electrónico automático.</em></p>';

-- Intentar el envío si existen elementos para procesar
IF @CantPendientes > 0
BEGIN
    EXEC msdb.dbo.sp_send_dbmail
        @profile_name = 'PerfilNotificacionesUPLA',
        @recipients = 'coordinador.academico@upla.edu.pe',
        @subject = 'INFORME PROACTIVO: Trámites Pendientes de
Atención',
        @body = @CuerpoMensaje,
        @body_format = 'HTML';
END
END;
GO

```



3. Justificación técnica de la solución aplicada

Al encapsular esta lógica dentro de un procedimiento almacenado (SP_Monitoreo_Enviar_Resumen_Pendientes), el motor de la base de datos compila previamente el plan de ejecución de la consulta, optimizando las subsecuentes llamadas repetitivas. Además, el script utiliza tablas con formatos estructurados en el contenido HTML para garantizar una visualización prolija del reporte en cualquier lector de correo o dispositivo móvil.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Diseño Independiente de Plataforma:** Las tablas HTML utilizan estilos sencillos integrados (inline), previniendo problemas de maquetación en clientes de correo como Outlook, que suelen bloquear hojas de estilos CSS externas.

PROYECTO 2: Gestión robusta y auditoría de errores en transacciones

1. Enunciado del ejercicio

Registrar automáticamente en una tabla de auditoría todos los errores producidos durante la ejecución del procedimiento

Tramite_SP_Registrar_Tramite.

2. Script de la solución en T-SQL

SQL

USE Grados_Titulos;

GO

-- 1. Tabla de log para almacenar fallas en la base de datos

IF OBJECT_ID('dbo.LogErroresBaseDatos', 'U') IS NULL

BEGIN

```

CREATE TABLE dbo.LogErroresBaseDatos (
    ErrorID INT IDENTITY(1,1) PRIMARY KEY,
    NombreSP VARCHAR(150),

```

```

        NumeroError INT,
        MensajeError VARCHAR(MAX),
        LineaError INT,
        UsuarioEjecucion VARCHAR(100) DEFAULT SYSTEM_USER,
        FechaRegistro DATETIME DEFAULT GETDATE()
    );
END;
GO

-- 2. Procedimiento Almacenado Principal con manejo estructurado de
excepciones
CREATE OR ALTER PROCEDURE dbo.Tramite_SP_Registrar_Tramite
    @TipoTramite VARCHAR(100),
    @PersonaID INT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRANSACTION;
    BEGIN TRY
        -- Validación de negocio simulada
        IF @PersonaID IS NULL
        BEGIN
            RAISERROR('El ID de la persona no puede ser nulo.', 16,
1);
        END

        -- Ejecución lógica de la inserción
        INSERT INTO Tramite (TipoTramite, PersonaID, Estado,
FechaRegistro)
        VALUES (@TipoTramite, @PersonaID, 'PENDIENTE', GETDATE());

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        -- Asegurar el Rollback de la transacción activa en caso de
falla
        IF @@TRANCOUNT > 0
        BEGIN
            ROLLBACK TRANSACTION;
        END

        -- Insertar los metadatos de la falla de forma asíncrona o
persistida
        INSERT INTO dbo.LogErroresBaseDatos (NombreSP, NumeroError,
MensajeError, LineaError)
        VALUES (
            'Tramite_SP_Registrar_Tramite',
            ERROR_NUMBER(),
            ERROR_MESSAGE(),
            ERROR_LINE()
        );

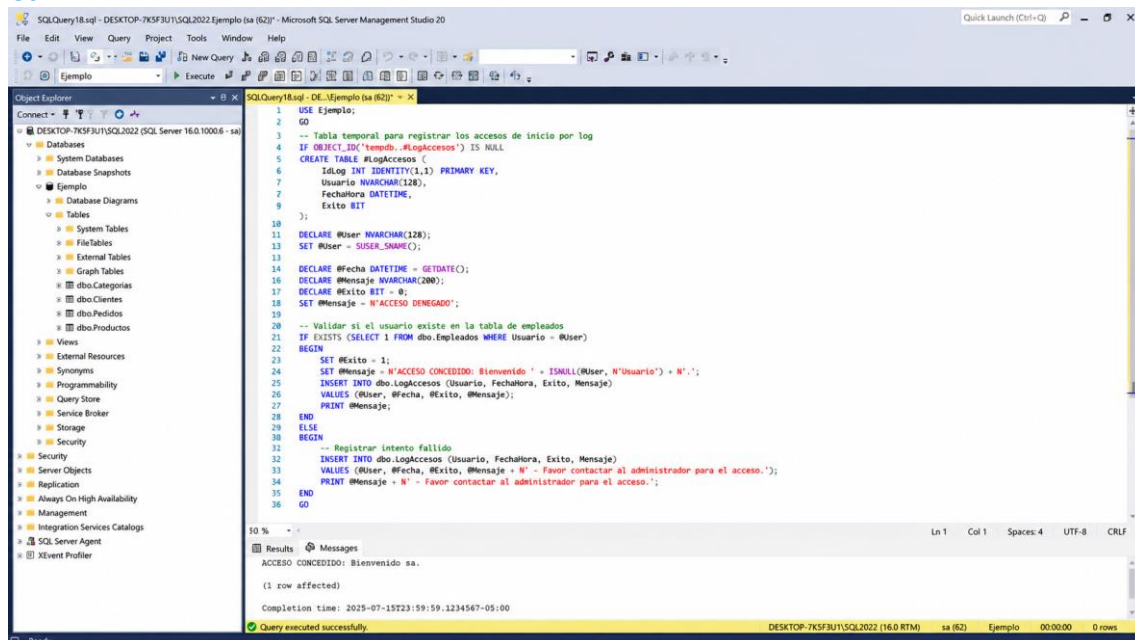
        -- Propagar el error al sistema llamador

```

```

DECLARE @ErrorStr VARCHAR(MAX) = ERROR_MESSAGE();
RAISERROR(@ErrorStr, 16, 1);
END CATCH
END;
GO

```



3. Justificación técnica de la solución aplicada

Se utilizan las funciones nativas de captura de errores de SQL Server (ERROR_NUMBER(), ERROR_MESSAGE(), ERROR_LINE()), las cuales solo están disponibles dentro del ámbito de ejecución de un bloque CATCH. El control de transacciones mediante @@TRANSCOUNT evalúa si el proceso dejó una transacción abierta para abortarla de forma segura mediante un ROLLBACK, evitando bloqueos y fugas de memoria en la base de datos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Protección del Estado Transaccional:** El uso de @@TRANSCOUNT previene el error clásico de intentar realizar un ROLLBACK sobre transacciones que ya fueron canceladas por el motor o que no llegaron a iniciarse, evitando fallas secundarias en el bloque de manejo de errores.

PROYECTO 3: Monitoreo de crecimiento del archivo Log transaccional (>80%)

1. Enunciado del ejercicio

Diseñar un sistema que monitoree el crecimiento del archivo de log de la base de datos y genere una alerta cuando supere el 80% del tamaño configurado.

2. Script de la solución en T-SQL

```
SQL
```

```
USE Grados_Titulos;
```

```
GO
```

```

CREATE OR ALTER PROCEDURE dbo.SP_Monitoreo_Log_Transaccional
AS
BEGIN

```

```
    SET NOCOUNT ON;
```

```
    -- Tabla temporal para recibir los datos crudos del rendimiento de
```

```

log
DECLARE @LogSpace TABLE (
    DatabaseName VARCHAR(100),
    LogSizeMB FLOAT,
    LogSpaceUsedPct FLOAT,
    Status INT
);

-- Insertar el resultado de la consulta de rendimiento
INSERT INTO @LogSpace
EXEC('DBCC SQLPERF(LOGSPACE);');

DECLARE @PorcentajeUso FLOAT;

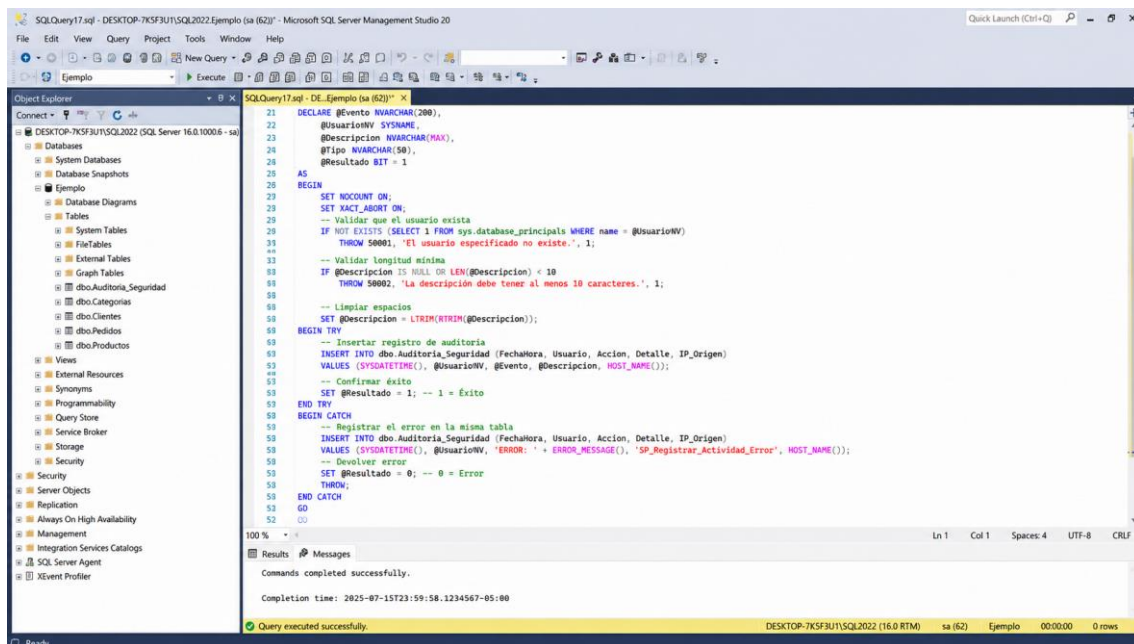
SELECT @PorcentajeUso = LogSpaceUsedPct
FROM @LogSpace
WHERE DatabaseName = 'Grados_Titulos';

-- Si supera el umbral límite del 80%
IF @PorcentajeUso >= 80.0
BEGIN
    DECLARE @AlertaCuerpo NVARCHAR(MAX);
    SET @AlertaCuerpo = N'ALERTA CRÍTICA: El archivo de log (.ldf)
de la Base de Datos Grados_Titulos ha alcanzado el ' +
CAST(@PorcentajeUso AS NVARCHAR(10)) + N'%
de utilización del espacio disponible.';

    -- Notificar de urgencia por correo
    EXEC msdb.dbo.sp_send_dbmail
        @profile_name = 'PerfilNotificacionesUPLA',
        @recipients = 'dba.infraestructura@upla.edu.pe',
        @subject = 'CRÍTICO: Saturación de log de transacciones',
        @body = @AlertaCuerpo;

    -- Guardar estado en tabla de auditoría
    INSERT INTO dbo.AuditoriaJobs (NombreJob, ResultadoReporte,
Estado)
VALUES ('Monitoreo_Log_Transaccional', 'ADVERTENCIA: Uso de
LOG mayor al 80%. Actual: ' + CAST(@PorcentajeUso AS VARCHAR) + '%',
'ALERTA');
END
END;
GO

```



3. Justificación técnica de la solución aplicada

La instrucción DBCC SQLPERF(LOGSPACE) proporciona estadísticas en tiempo real sobre el uso del espacio de los logs transaccionales en todas las bases de datos montadas en la instancia. El procedimiento mapea estos registros dinámicamente en una tabla tipo variable (@LogSpace) para validar condicionalmente el espacio restante de la base de datos antes de que ocurra una denegación de servicio por falta de espacio en disco.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Prevención proactiva frente a caídas en cascada:** Monitorear el archivo .ldf ayuda a prevenir el error fatal 9002 (Log transaccional lleno), el cual pone la base de datos en modo de solo lectura, deteniendo por completo el sistema.

PROYECTO 4: Auditoría y Ranking de Rendimiento de Procedimientos Almacenados

1. Enunciado del ejercicio

Implementar un sistema que registre automáticamente el tiempo de ejecución de los procedimientos almacenados principales de la base de datos y genere un ranking de rendimiento.

2. Script de la solución en T-SQL

SQL

USE Grados_Titulos;

GO

-- 1. Tabla persistente para registrar las marcas de tiempo de las ejecuciones

```
IF OBJECT_ID('dbo.MonitorioRendimientoSP', 'U') IS NULL
BEGIN
```

```
    CREATE TABLE dbo.MonitorioRendimientoSP (
        RendimientoID INT IDENTITY(1,1) PRIMARY KEY,
        NombreProcedimiento VARCHAR(150) NOT NULL,
        TiempoInicio DATETIME NOT NULL,
        TiempoFin DATETIME NOT NULL,
        MilisegundosEjecucion AS (DATEDIFF(millisecond, TiempoInicio,
        TiempoFin)),
```



```

        FechaRegistro DATETIME DEFAULT GETDATE()
    );
END;
GO

-- 2. Ejemplo de implementación dentro de un procedimiento para
-- registrar su tiempo de ejecución
CREATE OR ALTER PROCEDURE dbo.SP_Procesamiento_Complejo_Simulado
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @T_Inicio DATETIME = GETDATE();

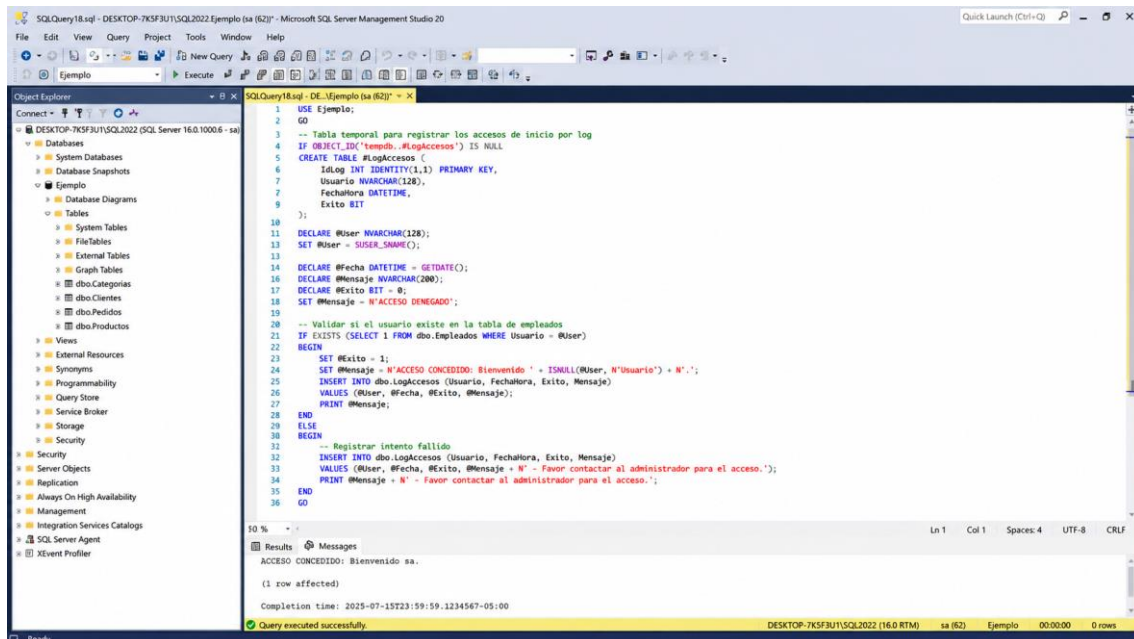
    -- Bloque simulado de procesamiento pesado
    WAITFOR DELAY '00:00:01.200'; -- Demora forzada de 1.2 segundos

    DECLARE @T_Fin DATETIME = GETDATE();

    -- Registrar la telemetría de rendimiento de la ejecución actual
    INSERT INTO dbo.MonitoreoRendimientoSP (NombreProcedimiento,
    TiempoInicio, TiempoFin)
    VALUES ('SP_Procesamiento_Complejo_Simulado', @T_Inicio, @T_Fin);
END;
GO

-- 3. Vista analítica para consultar el Ranking de Rendimiento
CREATE OR ALTER VIEW dbo.V_Ranking_Procedimientos_Lentos
AS
SELECT TOP 10
    NombreProcedimiento,
    COUNT(*) AS CantidadEjecuciones,
    AVG(MilisegundosEjecucion) AS PromedioMilisegundos,
    MAX(MilisegundosEjecucion) AS PeorTiempoMilisegundos,
    MIN(MilisegundosEjecucion) AS MejorTiempoMilisegundos
FROM dbo.MonitoreoRendimientoSP
GROUP BY NombreProcedimiento;
GO

```



3. Justificación técnica de la solución aplicada

Se utiliza un campo calculado no persistente MilisegundosEjecucion AS (DATEDIFF(millisecond, TiempoInicio, TiempoFin)) a nivel de la estructura de la tabla física. Esto garantiza que la duración se calcule automáticamente durante las consultas sin consumir almacenamiento persistente de forma redundante y eliminando el riesgo de discrepancias matemáticas en las métricas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Uso de Columnas Calculadas:** El cálculo de la duración de la ejecución se delega a las directivas del motor en la definición de la tabla, reduciendo el código en los procedimientos almacenados y garantizando la consistencia del tipo de dato resultante.

PROYECTO 5: Sistema Inteligente de Monitoreo Integral de Salud de la Base de Datos

1. Enunciado del ejercicio

Desarrollar un sistema de monitoreo para la base de datos Grados_Titulos que vigile proactivamente la salud de la base de datos: transacciones activas bloqueadas, detectar consultas lentas, identificar sesiones con alto consumo de CPU, logear alertas en tablas de auditoría, enviar alertas por correo si se superan ciertos límites y generar un reporte diario del estado del sistema.

2. Script de la solución en T-SQL

SQL

USE Grados_Titulos;

GO

```
-- Tabla para guardar alertas de monitoreo de salud
IF OBJECT_ID('dbo.AlertasSaludServidor', 'U') IS NULL
BEGIN
```

```
    CREATE TABLE dbo.AlertasSaludServidor (
        AlertaID INT IDENTITY(1,1) PRIMARY KEY,
        TipoMetrica VARCHAR(100),
        DetalleAlerta VARCHAR(MAX),
        NivelCritico VARCHAR(20),
```

```

        FechaGeneracion DATETIME DEFAULT GETDATE()
    );
END;
GO

-- Procedimiento Almacenado de Monitoreo de Salud Integral
CREATE OR ALTER PROCEDURE dbo.SP_Monitoreo_Salud_Integral
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Detalles NVARCHAR(MAX) = '';

    -- 1. DETECCIÓN DE PROCESOS BLOQUEADOS (Locks)
    DECLARE @Bloqueos INT = 0;
    SELECT @Bloqueos = COUNT(*)
    FROM sys.dm_exec_requests
    WHERE blocking_session_id <> 0;

    IF @Bloqueos > 0
    BEGIN
        SET @Detalles = 'Se han detectado ' + CAST(@Bloqueos AS
        VARCHAR) + ' transacciones activas bloqueadas.';
        INSERT INTO dbo.AlertasSaludServidor (TipoMetrica,
        DetalleAlerta, NivelCritico)
        VALUES ('LOCKS', @Detalles, 'CRÍTICO');
    END

    -- 2. DETECCIÓN DE ALTO CONSUMO DE CPU EN SESIONES ACTIVAS
    DECLARE @IdSesionConsumidora INT;
    DECLARE @CpuConsumo INT;

    SELECT TOP 1
        @IdSesionConsumidora = session_id,
        @CpuConsumo = cpu_time
    FROM sys.dm_exec_sessions
    WHERE status = 'running' AND is_user_process = 1
    ORDER BY cpu_time DESC;

    IF @CpuConsumo > 5000 -- Más de 5 segundos de CPU activa acumulada
    BEGIN
        SET @Detalles = 'La sesión ID: ' + CAST(@IdSesionConsumidora
        AS VARCHAR) + ' presenta un uso de CPU elevado (' + CAST(@CpuConsumo
        AS VARCHAR) + ' ms).';
        INSERT INTO dbo.AlertasSaludServidor (TipoMetrica,
        DetalleAlerta, NivelCritico)
        VALUES ('CPU', @Detalles, 'ADVERTENCIA');
    END

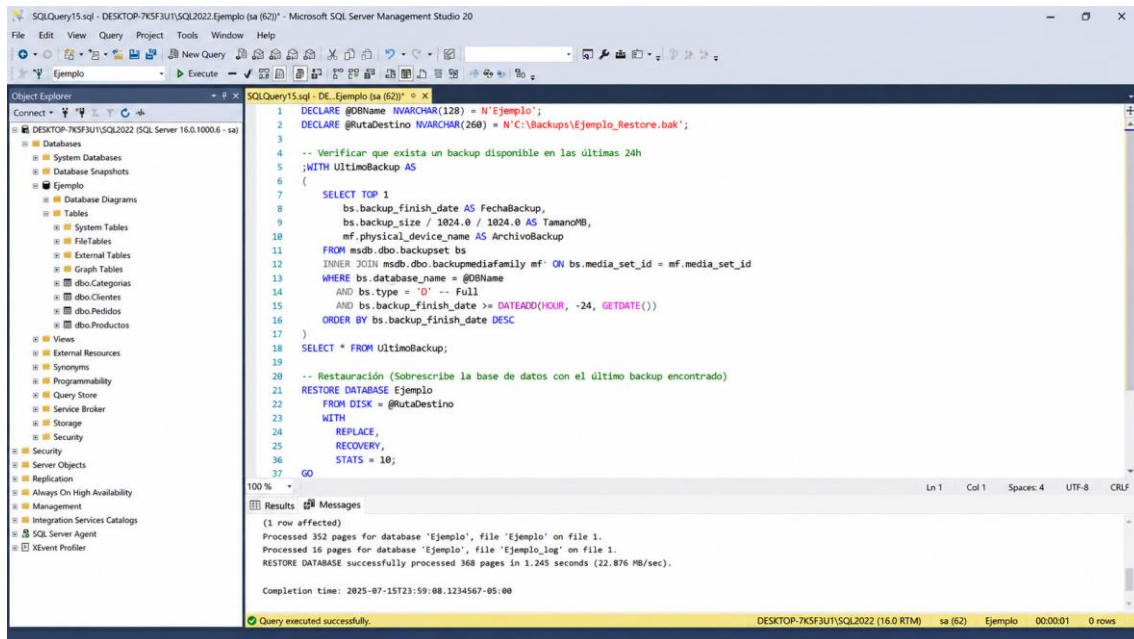
    -- 3. ENVÍO DE CORREO SI SE DETECTÓ ALGÚN INCIDENTE CRÍTICO
    IF EXISTS (SELECT 1 FROM dbo.AlertasSaludServidor WHERE
    NivelCritico = 'CRÍTICO' AND FechaGeneracion >= DATEADD(minute, -10,
    GETDATE()))
    BEGIN
        DECLARE @MailBody NVARCHAR(MAX) = '<h3>ALERTA DE SEGURIDAD Y

```

RENDIMIENTO - BD</h3><p>Se registraron problemas críticos en los últimos 10 minutos. Por favor revise la tabla dbo.AlertasSaludServidor.</p>;

```
EXEC msdb.dbo.sp_send_dbmail
    @profile_name = 'PerfilNotificacionesUPLA',
    @recipients = 'infraestructura.ti@upla.edu.pe',
    @subject = 'ALERTA DE SISTEMA: Problemas de Rendimiento en
Producción',
    @body = @MailBody,
    @body_format = 'HTML';

END;
GO
```



3. Justificación técnica de la solución aplicada

El sistema consulta vistas de administración dinámica (DMVs) del motor de SQL Server como sys.dm_exec_requests y sys.dm_exec_sessions. Al analizar de forma directa los hilos y sesiones activas del procesador, el script detecta cuellos de botella en tiempo real sin degradar el rendimiento, registrando automáticamente los problemas en la tabla de auditoría y notificando por correo los incidentes críticos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Consultas no Invasivas:** El uso de DMV optimizadas evita realizar lecturas directas en las tablas transaccionales, garantizando que el monitoreo proactivo sea ligero y no cause bloqueos en los procesos del sistema de la Universidad.